Imperial College London

MEng Individual Project

Imperial College London

Department of Computing

VizER: Data Visualisation Based On Conceptual Modelling

Author:
Mohammed Abdus Samad Hannan

Supervisor:
Peter McBrien

Second Marker:
Holger Pirk

June 17, 2024

Abstract

There exists a wide range of data visualisation tools in the current landscape which offer the ability to select data and visualise it in interesting ways. However, most of these tools do not seem to take advantage of the conceptual model of the data in order to generate the best visualisations. As a result, domain experts and non-technical users struggle to create meaningful visualisations with current tools as they are more familiar with the conceptual model of their data rather than the tabular representations.

In this report, we propose an alternative approach to current industry standards, where we use a conceptual model to express the underlying relationships within the data and use this model to recommend the most suitable or feasible visualisations. We then discuss our implementation of the application VizER which enforces our proposed approach and creates an informative and meaningful experience for all users with any level of technical knowledge.

Acknowledgements

I would like to thank my supervisor, Dr. Peter McBrien, for providing me with his time, resources, and guidance throughout the entirety of this project. He has always made himself available for me and never left any of my questions unanswered. His feedback has allowed me to view my work from different perspectives and subsequently take it to the next level. His support has been nothing short of invaluable and I am forever grateful for his attitude and understanding towards me and my situation.

I would also like to thank all the participants of the user testing for giving me their time and effort to provide feedback on the application and ensure it was the best it could be.

Contents

Introduction

1.1 Motivation

In the current age of information, databases have allowed organisations to store their knowledge in repositories to serve as a backbone to their applications. Relational databases add shape and form to this by introducing tabular structures, relationships between entities, and standardised querying to enforce data integrity and easy data manipulation. The use of data analysis for the purpose of making supported and informed decisions is common across many industries, such as financial and marketing analysis, customer relationship management, as well as applications in education and healthcare. Data analysis uses statistical methods and techniques to draw meaningful conclusions that can be used to support the decision making of any agent [1]. Therein lies the necessity for companies to employ data analysts who have the ability to identify trends, patterns and anomalies from significant volumes of data.
From raw data alone, this is no simple task. Considering the scale and effort required to obtain valuable insights from large volumes of data, there is a clear need for powerful visualisation tools to aid analysts in the process [2]. Such data visualisers must be capable and effective in all aspects of exploratory visual analysis, for both open exploration of the data and more targeted and focused data analysis [3]. This introduces a new level of efficiency to the work of a data analyst, and automates part, if not all, of the process - examining data at vast scales and producing useful visualisations to support the business in making data-driven decisions.
While the industry does have widely used tools for this purpose such as Tableau [4], the approach for recommending the best visualisations to the analyst is often based on specific metrics measuring how useful a visualisation is in producing a valuable insight. The focus on the tabular presentations of the data make it difficult for domain experts, who understand more about the conceptual model of the data, to find the visualisations that match their exploratory visual needs [5]. An alternative approach to current solutions would be to utilise the full knowledge of the relational schema to select the most useful or feasible visualisations for that data [5]. In turn, this approach could widen the potential user base for data visualisation tools due to being easier to understand and use from a non-technical standpoint.

1.1.1 Motivating Example

Using Tableau, the industry standard for data visualisation, and Imperial's Mondial database, we show how current industry tools can fail to produce meaningful visualisations. We use the `encompasses` table for this example (Figure 1.1), attempting to visualise percentage-wise how much each country's land area is located in each continent.

After selecting each column of interest (country, continent, percentage), we receive the recommended visualisation from Tableau shown in Gallery, Figure A.1. We know this is the recommended visualisation type there is a orange-red outline around the visualisation type Tableau's Show Me window (Gallery, Figure A.2). From inspecting the symbol map, we see that this is not a suitable visualisation for our task, neither is it something we can analyse

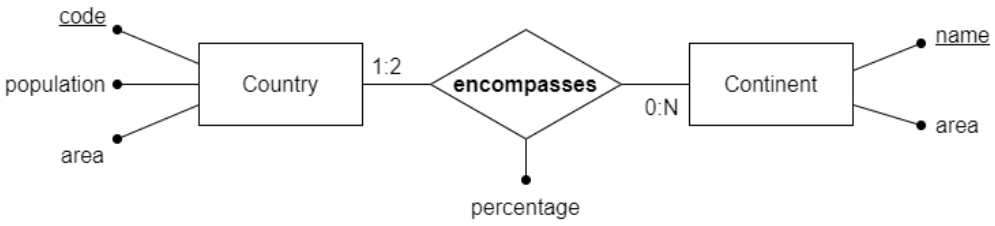


Figure 1.1: ER schema showing the many-many relationship `encompasses`

properly and draw insights from. The data is clustered and Tableau provides no interactive features that would allow us to zoom into the map. Looking through the other visualisation options suggested by Tableau's Show Me, none provide an adequate visual representation of the data. This is because Tableau does not have advanced visualisations which have the capacity to express complex relationships such as those with a many-many cardinality. Analysts can end up spending substantial time building advanced visualisations like sankey diagrams (which would be the correct view for this example, see Gallery Figure A.15) as custom visualisations in Tableau, often failing to do so well [6]. Lastly, the lack of interactiveness in Tableau's visualisations means that for highly concentrated points such as the visualisation of data from every country, users are unable to zoom, pan or navigate the graph in any way that will provide them with a in-depth, detailed view of its information. Therefore, the pain points we will discuss and attempt to tackle in our approach are:

- Lack of correctly recommended visualisations that utilise a conceptual model to provide users with a better understanding of their data.
- Lack of complex and interactive visualisations that provide users with an overall better view of their information.

1.2 Objectives

In this project, we present a data visualisation tool VizER that utilises the conceptual model of the data to recommend visualisations. We use an Entity-Relationship model of the relational schema to represent the underlying relationships within the data. We also introduce a set of visualisation schema patterns that distinguish each group of commonly-used data visualisations, and create mappings between these visualisations and the schema patterns with the use of the conceptual model [5]. The tool uses these schema patterns to provide the user with meaningful, interesting, and interactive visualisations.
We also present a secondary mode to VizER as an extension to this project which promotes a different approach of utilising the mappings between schema patterns and visualisation groups outlined by McBrien [5]. In this mode, the tool allows the user to explore various permutations of their data to produce a particular visualisation type. This allows the user to gain an in-depth understanding of their data's conceptual model and which parts of their data match each of the schema patterns. This way, the user can approach the main mode of the tool with a better idea of their analysis goals.
Ideally, this solution should not require the steep learning curve we find with current industry tools such as Tableau, and can be enjoyed by all users with access to a PostgreSQL relational database. However, we have created the application in an extensible and scalable way such that it can be extended upon to improve the user's experience and capabilities within VizER.

Background

In this section, we discuss the main two aspects of this project: the visualisation of data, and the conceptual modelling of relational databases. We also introduce various related work linked to the topic of this project, to discover what approaches have already been taken by others.

2.1 Data Visualisation

2.1.1 The Importance Of Visualisation

Ware elaborates on the importance of data visualisation in [7] by describing the effect of learning through visual displays. With the most effective form of communication between computer and human being via visual displays, data visualisation allows analysts to comprehend huge amounts of well-presented data, enabling them to easily interpret large quantities of information. Ware also describes the stages of data visualisation, shown in Figure 2.1 below.

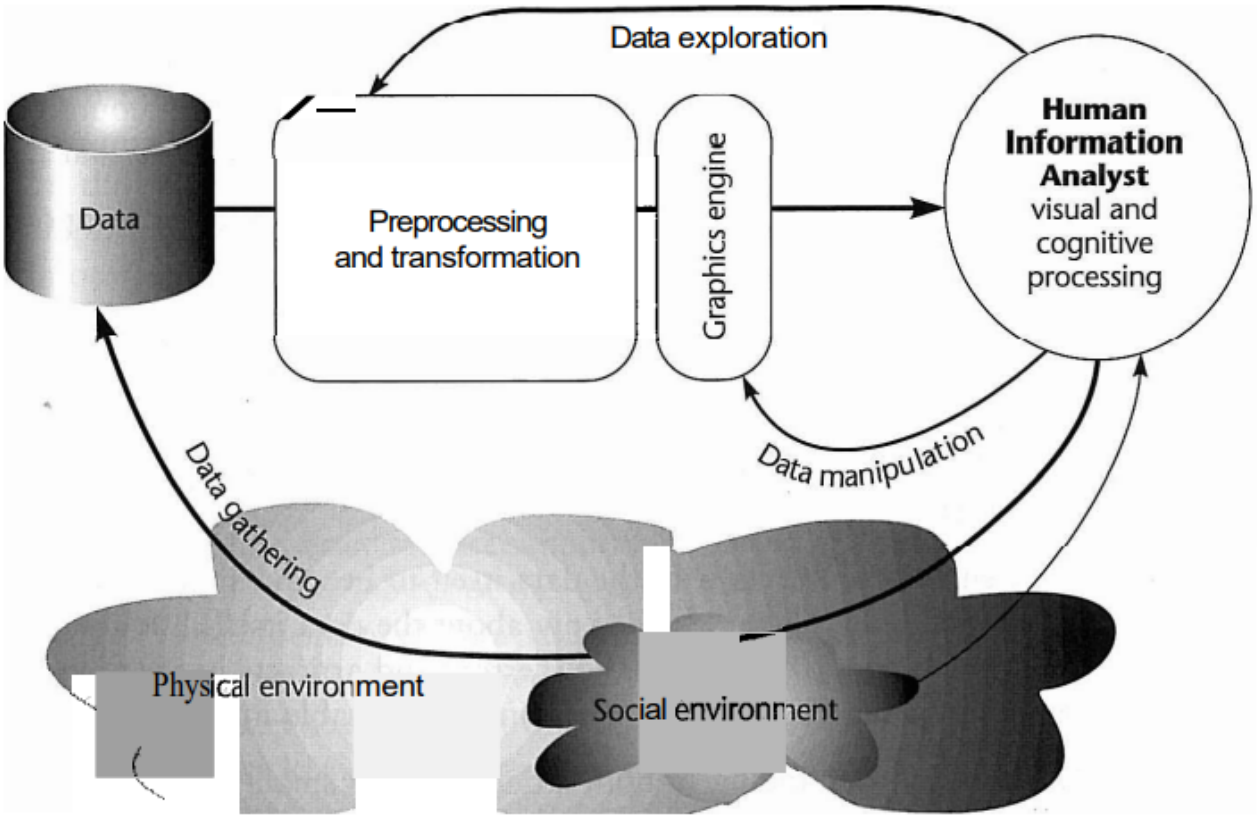


Figure 2.1: A schematic diagram of the data visualisation process [7]

The four stages outlined in this process are:

- Collecting and storing the data
- Preprocessing the data to make it more understandable
- Producing a visual display via the display hardware and graphics algorithms
- The analyst who will receive the visual information

The physical environment is defined as a source of data, whereas the social environment determines what data to collect and how the perceiver of the visualisation (the analyst) should interpret it. We see from the second stage that transformations can be applied to help the analyst understand the data better - an example Ware gives is selecting the most interesting parameter ranges. This links to McBrien's approach in [5] (the basis for this project) where mappings between data visualisations and visualisation schema patterns can be used to perform similar transformations such as extracting data, drill-down, pivot etc.
Interactive visualisations are also discussed in [7] where they are described as interfaces between the human visual system with its pattern finding capabilities, and the computer containing vast computational power and information resources. By improving these interfaces, we improve the system as a whole. Muzner also discusses the importance of interaction in [8] , specifically the implementation of interactively changing visual displays. With large datasets, humans and displays are extremely hindered in information exchange without interactivity.

2.1.2 The Grammar Of Graphics

Wilkinson's Grammar of Graphics [9] introduces a system of seven classes that can be used to generate any visualisation. These seven components are orthogonal, meaning they are distinct and can be adjusted independently of each other, thus we can generate a huge range of different visualisation types [10]. Wilkinson makes the following claims:

- Most, if not all, known charts can be generated by this simple system, including chart types that are yet to be discovered [10].
- The system is not just a tool for generating visualisations, but also a way of understanding the decisions we make when constructing statistical graphs, charts, and visualisations. The system is computational but based on mathematical principles which represent functions as data [10].

The diagram in Figure 2.2 below is a data-flow diagram, describing how a chart can be constructed in WIlkinson's system - more specifically, how a chain of mappings can translate a dataset into a meaningful graphic. The specific ordering of the subtasks shown below must be enforced as changes to this ordering can result in the generation of meaningless graphics [10].

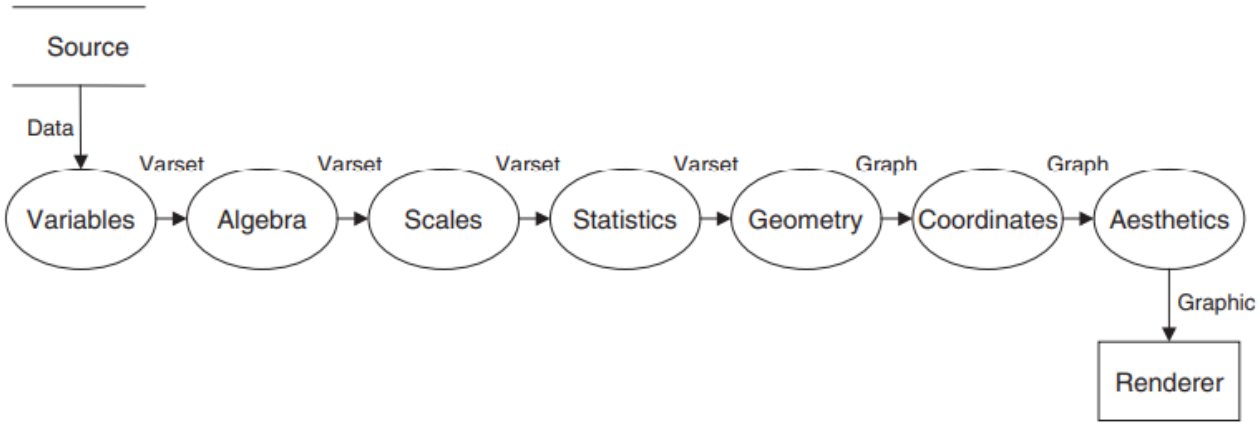


Figure 2.2: A data-flow diagram shown in [9] describing the chain of components

In [10], the process is summarised:

- The **Variable** class maps the input data from the source to a set of variables, also called a varset.
- In both the **Algebra** and the **Scales** class, various transformations are performed on the varset.
- The **Statistics** class takes in a varset, computes statistical summaries and returns another varset.
- The **Geometry** class takes a statistical graph and produces a geometric graph using graphing functions.
- The **Coordinates** class takes the geometric set and embeds it into a coordinate space, with the most popular chart types using a Cartesian coordinate space.
- The **Aesthetics** class is the final stage, which uses aesthetic functions to map the graph to a type of visualisation. This graphic is then rendered and displayed to the user.

2.1.3 Visualisation Specification Languages

Many different visualisation specification languages exist in the data visualisation landscape, each with differing levels of detail and functionalities. A lot of these are implementations of the Grammar of Graphics or some variation of it.

GPL

GPL is a direct implementation of the Grammar of Graphics. It is a language used in the IBM SPSS Statistics software for creating graphs [11]. The aim of GPL is to be a simple grammar that can be used to construct any graph, without the need for graph-specific commands. GPL consists of blocks of statements like in Figure 2.3, where each statement controls a specific aspect of the graph e.g. source data, graphic elements like points and lines, and coordinate systems. Statements start with a label; [11] defines each label type as:

- **SOURCE** - defining the file or dataset that is the source of data for the graph.
- **DATA** - assigning a column or field to a specific variable.

- **SCALE** - defining the scale used for the different dimensions of the graph and their range.
- **GUIDE** - controlling the aspects of the graph that help with interpreting it such as axis labels.
- **ELEMENT** - identifying the type of graphic to produce.

```
SOURCE: s = userSource(id("Employeedata"))
DATA: jobcat=col(source(s), name("jobcat"), unit.category())
DATA: salary=col(source(s), name("salary"))
SCALE: linear(dim(2), include(0))
GUIDE: axis(dim(2), label("Mean Salary"))
GUIDE: axis(dim(1), label("Job Category"))
ELEMENT: interval(position(summary.mean(jobcat*salary)))
```

Figure 2.3: An example block of GPL [11]

While this language does provide the analyst with a lot of control over the various aspects of the graph, it is clear that GPL is a low-level grammar which would be less easy to use and involve a steep learning curve.

ggplot2

Wickham introduces ggplot2, an open-source R package that can generate visualisations for statistical and data analysis [12]. The package implements the "Layered Grammar of Graphics", an extension of Wilkinson's Grammar of Graphics. In this extension of the grammar, Wickham outlines his approach for a different arrangement of Wilkinson's system based on constructing graphics using layers of data [13]. This differs from the original Grammar of Graphics in how it structures all its components and builds a hierarchy of defaults to make the plot-building process simpler. This layered grammar is also language-dependent on R. Figure 2.4 summarises the mappings of components between GPL, a direct implementation of Wilkinson's grammar, and ggplot2's layered grammar.

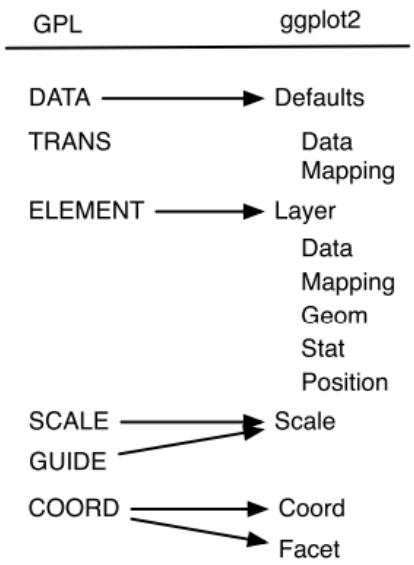


Figure 2.4: Mappings between GPL's grammar and ggplot2's layered grammar [13]

Wickham describes visualisation as just one part of the data analysis cycle (shown in Figure 2.5), and emphasises the importance of coupling visualisations in ggplot2 with alternative R packages which can handle data transformation and modelling. This means while ggplot2 allows analysts to give up low-level control for richer graphics, it is unable to produce interactive graphics.

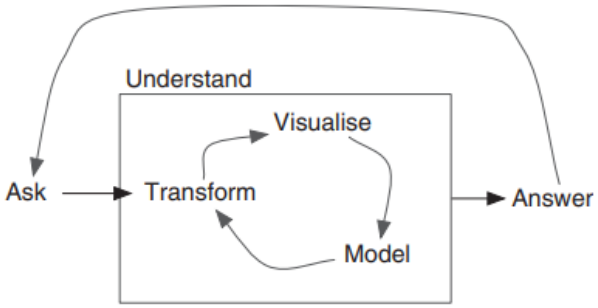


Figure 2.5: Wickham's description of the data analysis cycle [14]

Vega-Lite

Vega-Lite is a grammar created as a high-level abstraction of the low-level grammar Vega. Vega-Lite was implemented through the combination of the traditional Grammar of Graphics with a modern grammar of interaction which enables the quick and easy specification of interactive graphics like the ones show in Figure 2.6 [15]. Users are able to define the interactive features of their visualisations through the composition of selections. Satyanarayan defines selections as a tool that defines how inputs are handled, identifies what the points of interest are, and determines what to include. While not offering a wide range of visualisation types compared to Vega, Vega-Lite focuses on the generation of interactive visuals while keeping the specification concise [6]. Examples of visualisation types that cannot be built using Vega-Lite are treemaps, chord diagrams, and sankey diagrams.

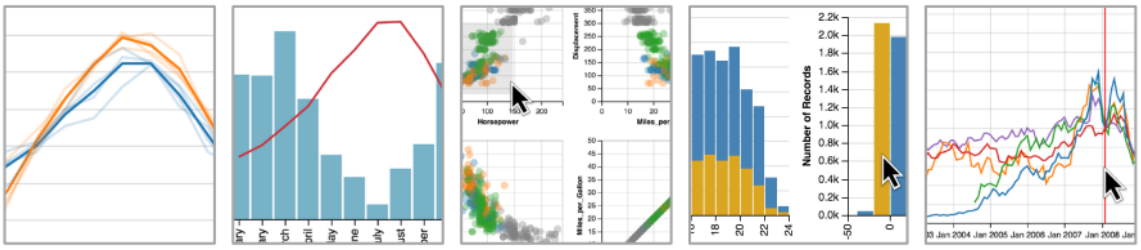


Figure 2.6: Examples of interactive visualisations produced using Vega-Lite [15]

2.2 Modelling Data

2.2.1 Entity-Relationship Model

The ER model separates data into entities and relationships, with entities being the targets of visualisation and relationships describing the structures and patterns that link entities together [7]. More formally:

- **Entities** are objects of interest, such as people or hurricanes. Entities are not limited to singular objects and when convenient, it is possible to represent a group of collected items as a single entity e.g. a school of fish [7].
- **Relationships**, also referred to as relations, are data structures that relate entities together. There is a range of relation types. Some practical examples include a wheel's relationship with a car, or a store and its customers, signifying that relations can be physical or conceptual. Causal and temporal relationships also exist, where the former is when one event is subsequently caused by another, and the latter describes a temporal difference between two events [7].

An example of an ER diagram is shown in Figure 2.7. In this diagram, entities are denoted by rectangles. The lines connecting entities are the relationships, represented by a diamond. The numbers beside a relationship (e.g. 0:N, 1:2) represent a cardinality constraint for that relationship. Attributes are represented by the lines protruding from an entity/relation with a circle at the end. An underlined attribute represents a primary key in the relational schema. A question mark beside an attribute's name denotes that attribute as optional.

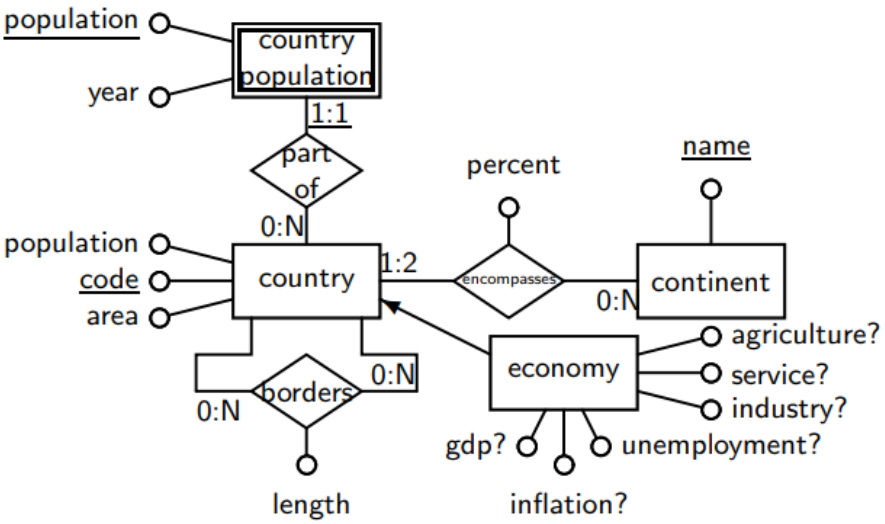


Figure 2.7: An example of an ER schema [5]

Attributes are defined as a property of either an entity or relation. They are often thought of as a descriptor for an entity or relation, like the colour of an apple or the duration of a journey [7]. The scales introduced by Stevens in [16] are one way of considering attribute quality in data visualisation. Stevens describes four levels of measurements:

1. **Nominal** scales are used to label objects where there is no ordering e.g. the various names of fruits.
2. **Ordinal** scales introduce the idea of rank-ordering e.g. the hardness of minerals. Here, while the numbers are arranged in a specific sequence, there is no benefit of calculating

statistical metrics such as mean or standard deviation as that implies something other than rank-ordering.

3. **Interval** scales, according to [16], describe almost all statistical measures, unless the measure has a true zero point. The scale takes note of the gap between each data point. Common examples include Centigrade and Fahrenheit.

4. **Ratio** scales are possible if all of the following are able to be determined: equality, rank-order, equality of intervals and equality of ratios. These scales do have an agreed-upon absolute zero point, even if it cannot be attained, such as the temperature Absolute Zero.

Most of the time, only three of Stevens's scales are used, with the most typical data types in visualisation being category data (similar to Stevens's nominal), integer data (similar to Stevens's ordinal) and real-number data (a combination of Stevens's interval and ratio scales). ER models are further discussed in McBrien and Poulovassilis's paper [5] where they discuss the approach to use visualisation schema patterns and relationships in the underlying ER model to recommend the best visualisations. As stated previously, this is achieved by creating mappings between the schema patterns and the range of visualisations. The table of mappings is shown in Figure 2.8 below.

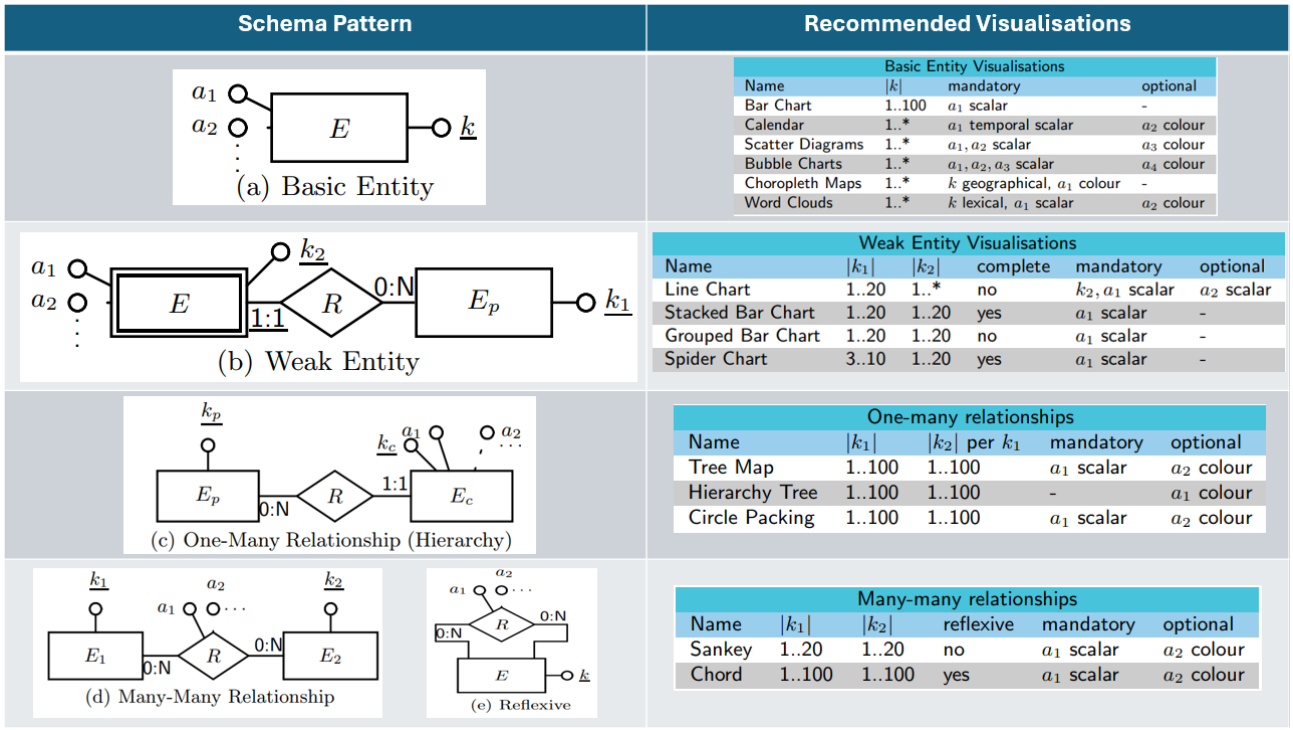


Figure 2.8: A table of mappings, as described in [5]

2.2.2 Reverse Engineering

In order to extract a conceptual model of our relational database like an Entity-Relationship model, we can use a reverse engineering method. Numerous approaches have already been documented which mainly involve the analysis of schema information and data manipulation statements within an application.
Premerlani states in [17] the rationale for this process is to enable the evolution of applications without the costly development of new software, nor the cost and effort of maintaining legacy software and adapting it to new uses. Premerlani and Blaha suggest the approach of creating

an object-oriented model to then convert into a relational schema. The object-oriented model consists of classes, generalisations and associations. Their approach is a seven-step process which starts with creating an initial object-oriented class model. The candidate keys are then determined, followed by refinement of the class model along with the discovery of generalisations and associations. Finally, transformations are applied to match the schema as close as possible to the database designer's original plan. An example of a class diagram generated by Premerlani and Blaha's process is shown in Figure 2.9. An important aspect of this reverse engineering approach is the incorporation of three information sources: the schema, the semantics of the application, and the patterns of data.

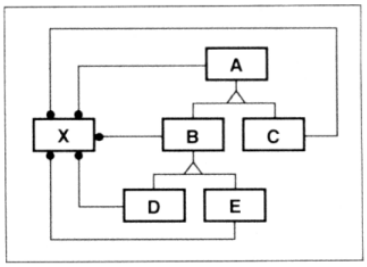


Figure 2.9: Class diagram, taken from [17]

Andersson proposes a reverse engineering process in [18] entailing the analysis of data manipulation statements found within the source code of an application that uses a relational DBMS (Database Management System). Andersson introduces the ERC+ model, an extension of the ER model, which adds multi-valued and complex objects as well as multi-instantiation operators "maybe" and "isa". An example diagram of an ERC+ model is shown in Figure 2.10. By analysing join conditions in queries and view definitions, Andersson's method is able to determine possible keys and references between tables in the schema. Initially, a connection diagram is created through a process of information retrieval where data manipulation statements in the application code are analysed. Andersson then introduces a list of translation rules, separated into node rules, link rules and refinement rules. By applying these rules (possibly applying the refinement rules multiple times to reach a complete solution), we are able to convert the previously created connection diagram to an ERC+ model.

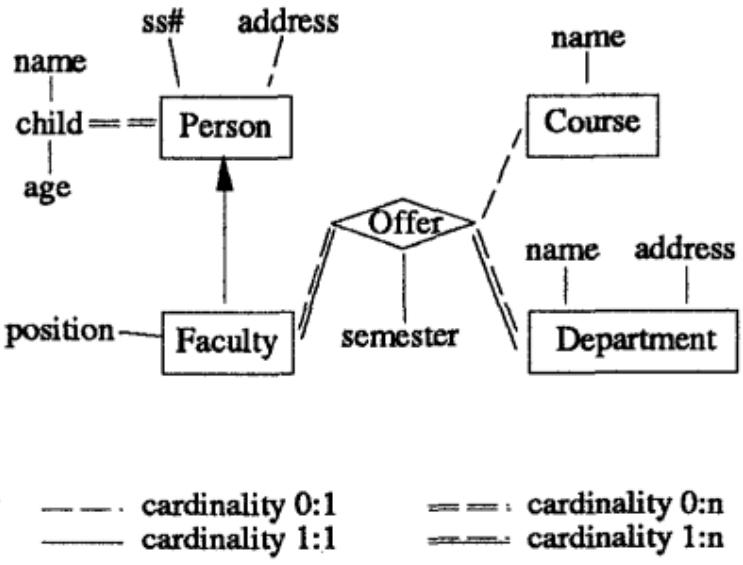


Figure 2.10: Diagram of an ERC+ schema model [18]

In [19], Petit discusses a major limitation of current reverse engineering approaches which is the requirement of the database to be in 3NF (Third Normal Form). For context, the different stages of normalisation in database design are:

- **FIRST NORMAL FORM (1NF)** - in this form, every attribute depends on the key. There are no elements or groups that are repeated [20].
- **SECOND NORMAL FORM (2NF)** - this form should have all the properties of 1NF, with the addition that any non-key attributes should be dependent on a single primary key [20].
- **THIRD NORMAL FORM (3NF)** - this form similarly has all the properties of 2NF, with the addition that non-key attributes cannot be dependent on any other non-key attribute [20].

To tackle this limitation, Petit outlines a method in [19] that removes the 3NF requirement and deals with the reverse engineering of denormalised relational schemas. For this approach, the only requirement is for the relational schema to be in 1NF. Petit's method, like other reverse engineering approaches, involves the analysis of equijoin queries in the application code. By deducing the inclusion dependencies, the functional dependencies and the hidden objects, Petit's method is able to normalise the 1NF schema to produce a 3NF schema. The schema will now follow the usual requirement and can be converted into a conceptual model.

2.3 Related Work

2.3.1 Tableau

Tableau is a commercial data visualisation and analysis system [21], described by themselves as the "market-leading choice for modern business intelligence" [22]. Tableau has an automatic data presentation feature called Show Me, described by Mackinlay as a set of commands and defaults in the user interface that allows the system to aid a user in the search for the best visualisation for their data [21]. Tableau's Show Me leverages VizQL, a high-level grammar - users use the drag-and-drop interface to add their variables to "shelves"; these actions are then translated into VizQL to allow quick visualisation creation. [23]. Show Me takes advantage of VizQL to automatically present the input data as a table of views. Mackinlay discusses the issue analysts face with data visualisation tools which is that while all analysts will have the required expertise of their domain, most do not have the skills or proficiency to leverage visualisation tools to design effective visual displays of information [21]. The Show Me functionality incorporates three different features into the user experience:

- **AUTOMATIC MARKS** - this feature analyses a VizQL expression and its row and column structure to determine the default mark type for a view [21] (where marks are points, lines, areas etc. [5]).
- **ADD TO SHEET** - this command can be used by the user to utilise the VizQL specification as well as the properties of fields to add more fields to a view using the best design practices [21].
- **SHOW ME and SHOW ME ALTERNATIVES** - these commands build views for multiple fields, incorporating a ranking system of views which is based on how well the designs apply best practices, and how familiar the visualisation type is to people. Show Me Alternatives creates a dialog box of a grid of the best possible visualisation choices, with

tooltips to explain why a specific visualisation choice is feasible/infeasible. An example of this is shown in Figure 2.11 below. Note that the ranking gives preference to views that present the data graphically, which gives text tables the lowest rank but also means the highest ranking visualisation types have many conditions to be met in order for that design to be an effective choice [21].

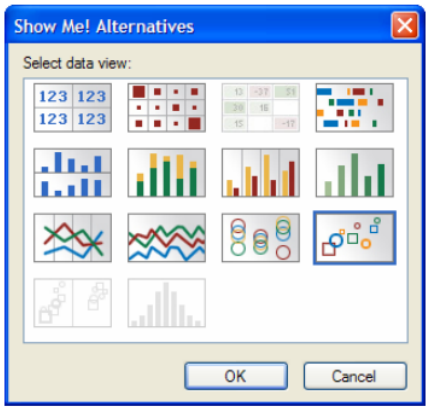


Figure 2.11: An example of the Show Me Alternatives feature of Tableau [21]

McBrien's paper [5] delves into the problems with this approach. An example visualisation using Tableau is shown in Figure 2.12. Here, we see that while the data is evidently skewed, the tool does not accommodate for this and as a result, produces a visualisation that would be difficult for an analyst to interpret or gather valuable insights from without further manual input. Current visualisation tools, also including D3.js and Google Charts also face this problem, where manual input is required to select appropriate encodings of the data and apply necessary transformations to make the data easier to interpret. The lack of interactiveness in particular makes this Tableau visualisation impossible to gain insights from - there is no way to unclutter the points.

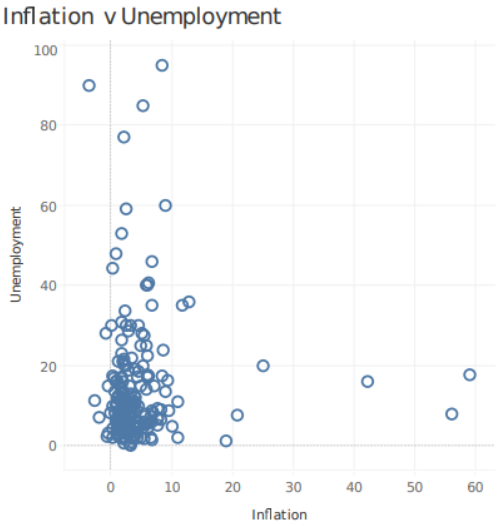


Figure 2.12: An example of an illegible Tableau visualisation [5]

2.3.2 AutoVis

Wills introduces AutoVis, an automatic visualisation system (AVS) that takes in content such as text, relational tables, hierarchies, streams, or images, and visualises that information in an appropriate manner [10]. The key aspect of this tool is that it focuses on **automatic** visualisation, not automated visualisation. The difference between the two is that the former entails the system automatically making decisions on what to be visualised, whereas the latter focuses more on automating the building process of charts, graphs and visualisations. The design of the AutoVis AVS system relies on three foundations:

- Wilkinson's Grammar of Graphics
- Scagnostics, a term for scatterplot diagnostics, involving techniques that allow interesting visuals to be extracted from pairs of variables [24].
- A modeler that uses the logic of statistical analysis as a foundation.

The AutoVis system is composed of the following features [10]:

1. An empty window as an interface where users drag and drop their data source onto.
2. A graphics generator that produces graphs without the need for any user definitions.
3. A statistical analyser which has the aim of protecting users from coming to false conclusions.
4. A pattern recogniser which is able to respond to the aspects of the visualisation such as density, shape, trend etc.

Wills describes the scenario in which AutoVis is most effective by explaining a situation where a data analyst is provided with data but does not know where to look to start the exploration process. Wills's approach has the aim of providing the analyst with some light so that they can begin the exploration process. There is emphasis placed on the tool providing enough light for the analyst to begin exploring, but not enough light that the analyst would be able to obtain conclusions about their data from AutoVis alone. Wills outlines the multiple processing stages involved in AutoVis's visualisation process. Two of these stages are analysis and prioritisation, where the former describes the use of steps from Wilkinson's Grammar of Graphics to analyse the structure of the data, and the latter entails prioritising the views that are the most "interesting" [10]. Scagnostics is used in the prioritising stage to identify useful scattlerplots. The three types of visualisations that AutoVis consider interesting are:

- **Notables** - the significant relationships that should be considered before modelling.
- **Exemplars** - the most typical joint distributions of points.
- **Anomalies** - the most unusual distributions of points.

We can see that while AutoVis is not the same type of system that we are building (we plan to build more of an **automated** visualisation system), there are certain stages in its procedure which we can relate to our approach, specifically the use of Wilkinson's Grammar of Graphics to analyse the data and produce visualisations, and the prioritisation of certain visualisations. The key difference will be our method of determining which views to prioritise by focusing on the underlying conceptual model of data.

2.3.3 SeeDB

SeeDB's structure is different to those of regular data visualisation tools. As opposed to being a standalone application, SeeDB is described as a wrapper that can be layered onto any RDBS (Relational Database System) [25]. The diagram Figure 2.13 shows a visual comparison between a traditional DBMS and SeeDB's system.

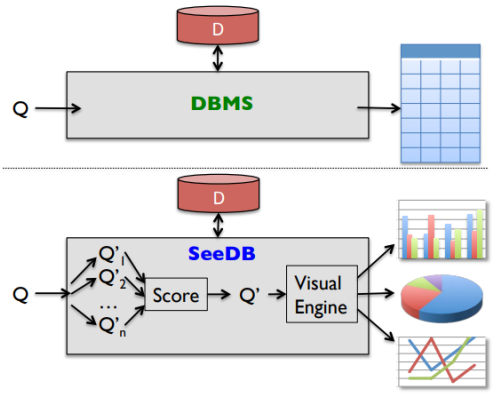


Figure 2.13: A comparison between a regular DBMS and SeeDB

Parameswaran describes SeeDB as a DBMS that partially automates the analysis workflow, specifically the laborious and tedious steps of constructing diverse visualisations of the data and then examining each view to determine which are the most interesting [26]. Given the user's query, the tool explores the space of all possible visualisations and uses a utility function to prune this search space and find interesting visualisations. The remaining visualisations in the pruned search space are then recommended to the analyst. The utility function is based on deviation from a comparison view, and determines how interesting a visualisation is. There are also steps put in place for those not too familiar with SQL to be able to use the system through the use of a query builder tool to generate queries via a form-based interface shown below [2]. Like AutoVis, this tool also has a step where visualisations are recommended based on their interestingness, but there seems to be no usage of the underlying relationships within the data at a conceptual level to include as part of the calculation.

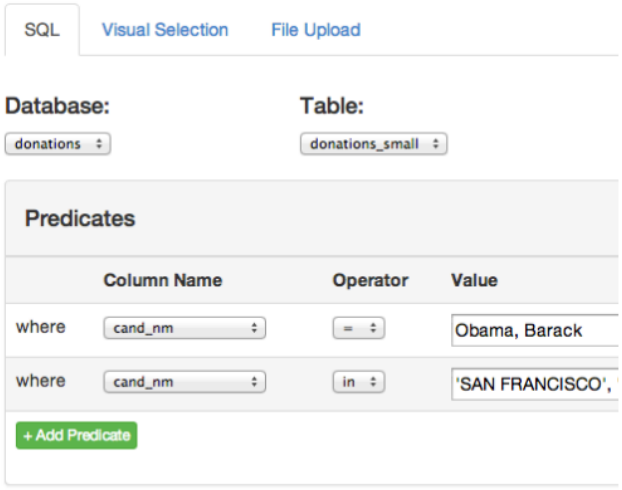


Figure 2.14: The query builder tool used in SeeDB [2]

2.3.4 Voyager and Voyager 2

The aim of the Voyager tool was to tackle the issue of manual specification of visualisations by providing an interactive navigation experience of a collection of automatically generated visualisations [23]. Wongsuphasawat explains that manual view specification involves the steps of choosing variables to examine, summarising the data with transformations, and coding in a high-level language or using a graphical interface to produce visualisations of the data. The approach taken by Voyager focuses more on open exploration of visualisations rather than the depth-first exploration strategies current tools are tailored toward. Voyager replaces specification with broad exploration, by providing a gallery of different recommended visualisations and allowing the user to interactively steer through the gallery. The tool uses Vega-Lite as the specification language to produce its visualisations and the Compass recommendation engine to enumerate, cluster and rank all the visualisations it produces. The architecture of the tool is shown in Figure 2.15.

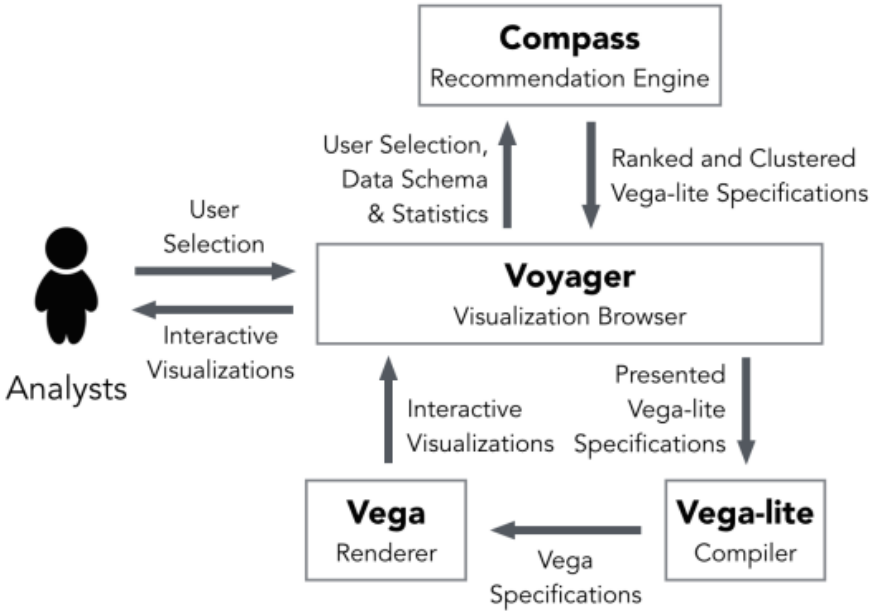


Figure 2.15: Architecture diagram of the Voyager tool

Since Vega-Lite is limited in the range of visualisation types it can produce and the lack of control it gives over high-level details, it is fair to say Voyager would also be limited in the types of graphs, charts, and visualisations it could produce.
Wongsuphawat later introduced Voyager 2, now able to provide support to analysts in both open-ended exploration (like the initial Voyager) and targeted analysis [3]. By outlining that the previous iteration of the tool did not sufficiently serve analysts who wanted a more focused analysis experience, Wongsuphawat added two new partial specification interfaces:

- **WILDCARDS** - these allowed users to change the properties of specification to generate multiple charts in parallel, leaving them in control of multiple views at the same time.
- **RELATED VIEWS** - this feature automatically suggested visualisations that were relevant to the currently user-focused chart, allowing the system to make more tailored recommendations to the user's specific analysis goals.

Voyager 2 was also given a different visualisation specification language, CompassQL. This was a generalisation of the Vega-Lite language that enabled the use of partial view specifications. This language is used to represent both specifications and recommendations. An example is shown in Figure 2.16 of how a user can use wildcards to consider alternative encodings of their specified view.

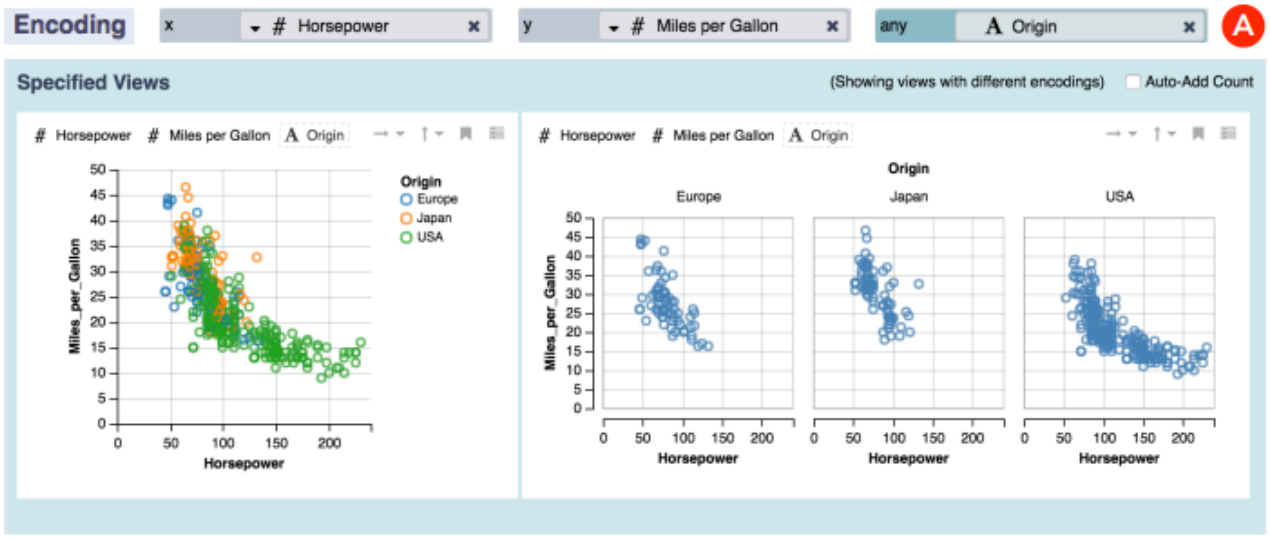


Figure 2.16: An example of the wildcard feature in Voyager 2 [3]

By going through the many related works to this project, we see that while some of these tools have similar approaches of producing and recommending visualisations of the user's data, none seem to show evidence of using the conceptual model of the data to determine which views to prioritise. What these tools provide the user with is elements to build a visualisation, but no guidance on which visualisation to build, or the correct way to put the elements together. Our aim for this project is to create a solution which achieves this through the use of schema pattern mappings outlined by McBrien in [5].

VizER: Approach

3.1 Application Design

Our main aim for this project is to create a data visualisation application that follows the approach outlined in [5], specifically with the use of visualisation schema patterns. This section goes into more detail about what our success criteria for this application is. Note that all examples given in this section are from Imperial's Mondial database.

3.1.1 Design Requirements

For the main mode of operation (known as the DATA-FIRST mode), we have the following requirements:

- To be able to connect to the user's database and display all tables and their columns to the user.
- To recommend the user specific visualisations based on their chosen columns and the corresponding schema pattern.
- To generate the user's chosen visualisation type for their selected data.

For the secondary mode of operation (the proposed extension known as the VIZ-FIRST mode), we have the following criteria:

- As previously stated, to connect to the user's database.
- To allow the user to select a specific chart type and provide guidance on what data can be visualised in this way (based on the schema patterns).
- To allow the user to explore all possible visualisations that can be produced from the selected chart type and data.

Both modes should also present information as to how their results were generated e.g. displaying information about the matched schema pattern and column data types.

3.1.2 Target Audience

As the application is a data visualiser, it should be developed in a way that requires minimal technical knowledge from the user. This would allow the application to be enjoyed by domain experts and non-technical users, not just data analysts. We do this by abstracting away most technical logic to the backend to achieve a more user-friendly experience e.g. allowing users to select columns via checkbox interface, thus knowledge of SQL is not required to as the query will be formulated and executed in the backend. As the secondary mode of operation (VIZ-FIRST mode) is designed as an exploratory experience, it requires even less technical knowledge

from the user. In this mode, steps are taken to ensure a visualisation of the chosen type can be produced, whereas with the main DATA-FIRST mode, there is the possibility of the user selecting columns that do not correspond to any schema pattern. For users unfamiliar on how to produce meaningful visualisations, they can use the VIZ-FIRST mode first to understand what combinations of columns correspond to a certain chart type and pattern, This will allow them to use the DATA-FIRST mode afterwards with less difficulty.

3.2 Mapping to Visualisation Schema Patterns

McBrien defines the four visualisation schema patterns in [5]. As discussed in Section 2.2.1, we use these schema patterns and relationships in the underlying ER model to recommend the best visualisations. This is done by utilising predefined mappings between each schema pattern and their distinct group of visualisation types. In this section, we discuss each visualisation schema pattern and how to identify them, and then how to determine suitable visualisations from each pattern's group of chart types.
For any of the patterns, the minimum requirement is that a primary key column and at least one attribute is chosen. This is because every chart type we support requires an independent variable and a dependent variable (excluding the hierarchy tree and network chart, which do not require an attribute). Recall that in our implementation, our mapping process has two rounds of pattern matching:

1. We perform column analysis on the primary and foreign key columns that have been chosen to identify the **visualisation schema pattern**.

2. We go through the schema pattern's list of visualisations and perform column analysis on all chosen columns (specifically looking at their data types) to determine which **chart types** within the pattern's group can be recommended to the user.

Basic Entity

A basic entity is defined as an entity which is not dependent on any other entity. It can be represented diagrammatically as E shown in Figure 3.1, with a single primary key k and one or more non-key attributes.

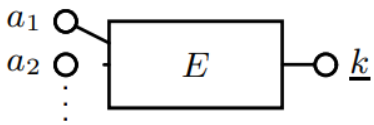


Figure 3.1: Diagram of a basic entity

This is the simplest pattern to identify. All we check for is that none of the columns chosen are a foreign key to another table as this would imply a dependency. An example of this is the `continent` table shown in Figure 3.2, where there is a single primary key column `name` and one attribute `area`. This table has no links to any other table in the Mondial database therefore we assume its representation in a conceptual ER model would be a basic entity.
McBrien also states the possibility for the key of a basic entity to be completely inherited as a foreign key to a different table [5]. An example of this is the `economy` table shown in Figure 3.3, where the primary key is inherited from the `country` entity. For this case, we must ensure that the **entire** primary key is inherited from a **single** entity which differs from the weak entity (where **part of** the primary key is a foreign key) and many-many relationship (where the entire primary key is composed of **two** foreign keys to other tables).

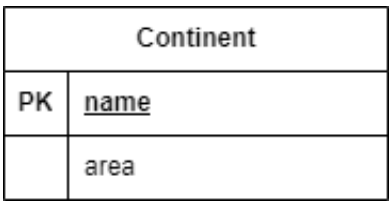


Figure 3.2: Example of a basic entity

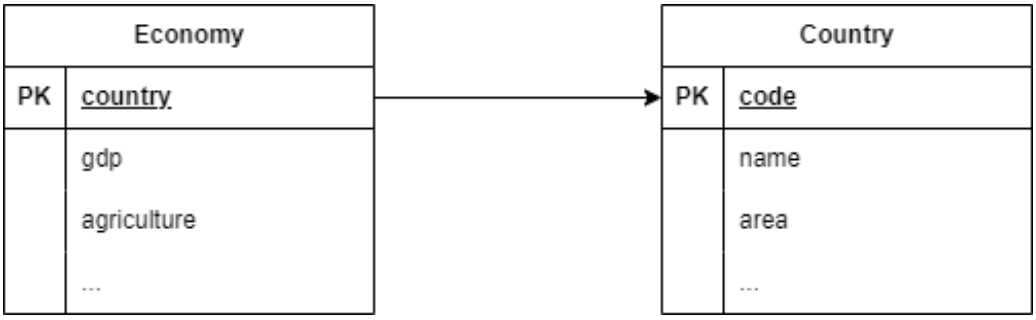


Figure 3.3: Example of a basic entity with inherited key

Weak Entity

A weak entity is defined as weak due to its dependency on a parent entity. The diagram in Figure 3.4 shows that the entity E has a compound key - part of its primary key is a foreign key k_1 to a parent entity E_p, and k_2 is unique to the child entity E.

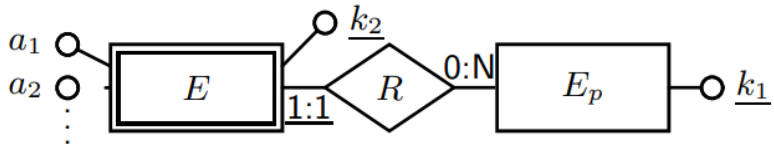


Figure 3.4: Diagram of a weak entity

We use this information about weak entities to identify the pattern in the user's column selection. If the user selects columns from a table where part of the primary key is a foreign key to a **single** parent table, we assume this table's representation in the conceptual ER model is a weak entity. An example of this pattern is the `country_population` table in Figure 3.5, which has two primary key columns - `country` (a foreign key to the `country` table), and `year` (specific to the `country_population` table).

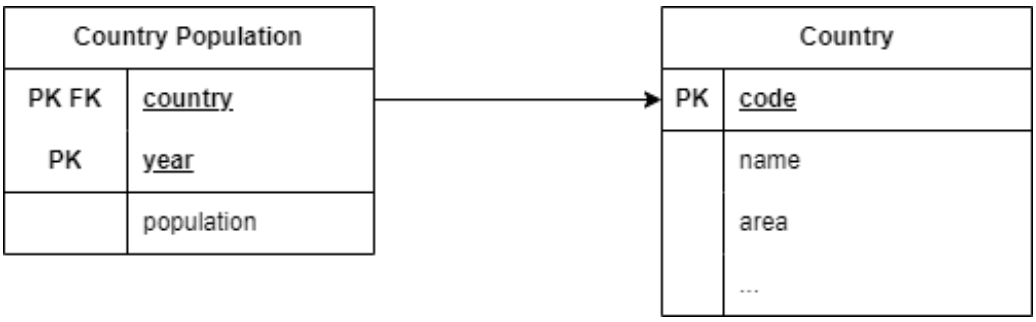


Figure 3.5: Example of a weak entity

One-Many Relationship

A one-many relationship is identified by the presence of a foreign key column to another entity. This is shown in Figure 3.6 where k_c represents the primary key of the child entity E_c and k_p represents a foreign key column to a parent entity E_p. Unlike the weak entity pattern, the foreign key k_p is not part of the child entity's primary key and therefore acts purely as a link to another table.

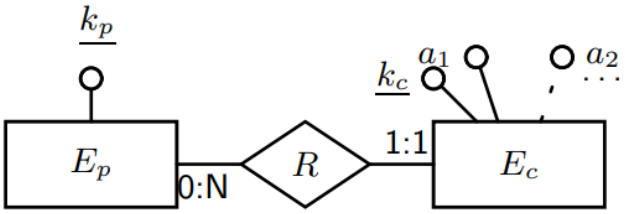


Figure 3.6: Diagram of a one-many relationship

When analysing the user's selection of columns, if the selection contains a **single** foreign key that is not part of the table's primary key, this corresponds to a one-many relationship in the underlying conceptual ER model. An example of this schema pattern is the `airport` table in Figure 3.7, with a foreign key to the `island` entity. It is obvious that an airport can only be situated on one island, whereas an island can have many airports on it - this affirms the one-many cardinality of the relationship between both entities. Therefore, if the user were to select the `island` column, the application should classify the user's selection as a one-many relationship. It is important to note that tables can have multiple foreign keys to different tables but a one-many relationship pattern can only be matched if the foreign key columns selected belong to a single parent entity. In the `airport` example, if the user were also to select the `city` column as well as `island`, the application would not classify the selection as a one-many relationship.

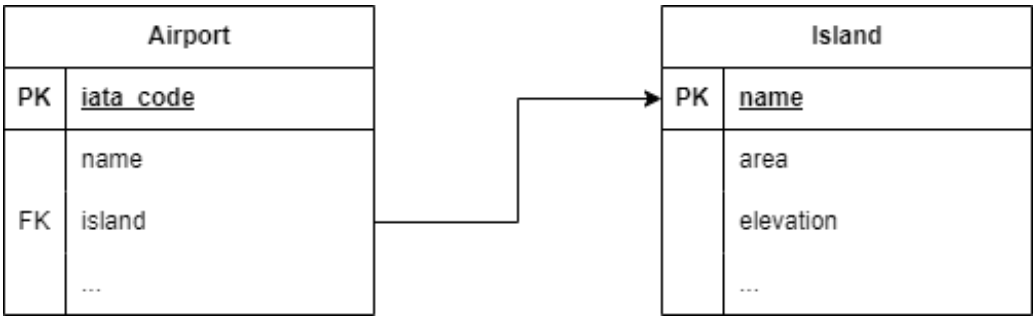


Figure 3.7: Example of a one-many relationship

Many-Many Relationship

For both weak entities and one-many relationships, the relationship between entities has been shown as column(s) in the child table acting as a foreign key to the parent table. With a many-many relationship, there exists a separate table R which is purely created from the relationship between entities E_1 and E_2. The primary key for this joint table R is composed of the primary keys of each parent table (shown in Figure 3.8 as k_1 and k_2). The table R also has non-key attributes which contain information about the relationship between the parent entities.

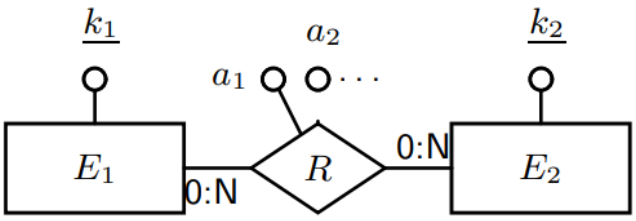


Figure 3.8: Diagram of a many-many relationship

To identify whether the user's chosen columns represent a many-many relationship, we analyse the primary key of the chosen table to determine if it consists of two foreign keys to separate tables. If so, we assume this table represents a joint table for a many-many relationship in the underlying conceptual ER model. An example of this is the `is_member` table in Figure 3.9. Here, we see that `is_member` acts as a joint table containing information about the relationship between `country` and `organization`.

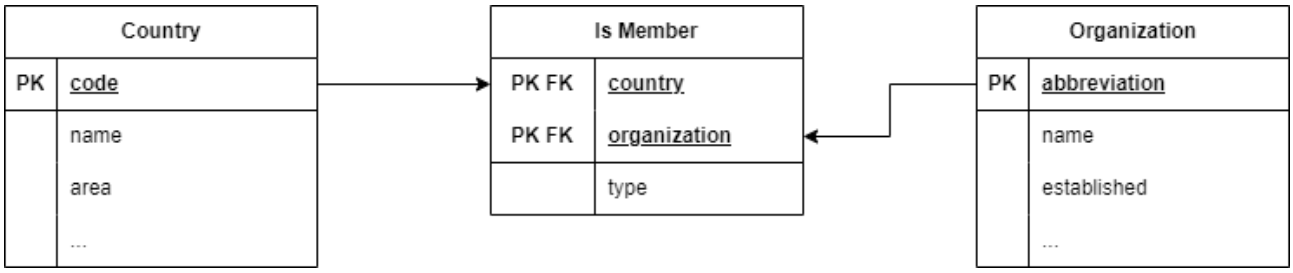


Figure 3.9: Example of a many-many relationship

Reflexive Many-Many Relationship

A special case for the many-many relationship is a reflexive many-many, shown in Figure 3.10. In this case, we do not have a relationship between two separate entities, but a relationship between an entity and itself with a many-many cardinality. The joint table still has two separate foreign key columns but both are linked to the same table (and aliased to avoid any overlap).

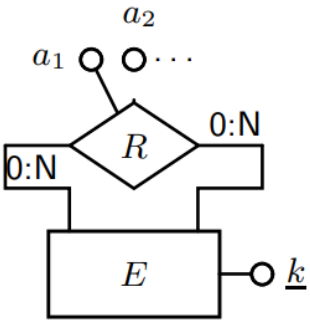


Figure 3.10: Diagram of a reflexive many-many relationship

We follow a similar process to the many-many relationship when identifying a reflexive many-many from the user's chosen columns. By additionally checking whether the two primary keys of the chosen table are foreign keys to the same table, we can determine if the many-many relationship is reflexive in nature. An example of this is the `borders` table in Figure 3.11, which represents a many-many relation `country` has with itself. The two primary key columns are aliased as `country1` and `country2` to avoid any naming ambiguity.

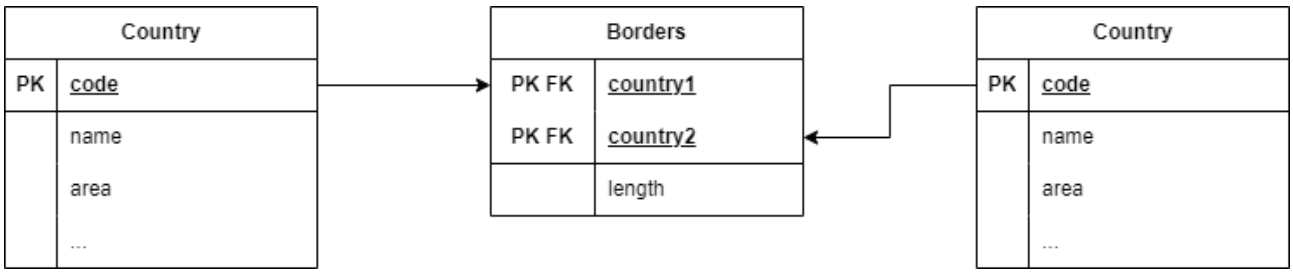


Figure 3.11: Example of a reflexive many-many relationship

3.3 Using SQL Data Types

As we move towards our second stage of pattern matching, we expand our focus from just the key columns in the user's selection to all chosen columns. Specifically, we examine the number of chosen attributes and the data types of all columns selected by the user. Since there are a large number of different data types, we group them to make the matching process easier. For this, McBrien introduces the concept of **scalar** and **discrete** dimensions, following a similar approach to Tableau [5]:

- **Discrete dimensions** usually have a small range of distinct values, often without a natural ordering - they are used to choose marks or vary a mark's channel (recall from Section 2.3.1 that a mark refers to points, lines, areas etc.).

- **Scalar dimensions** are continuous in nature, with a large range of values and a natural numeric ordering, such as integers, dates, timestamps, and floats - these usually represent a channel associated with a mark (where a channel is an aesthetic property of a mark such as its size, colour, length, or coordinates).

Interestingly, while colour is an example of a channel, and scalar dimensions are used to represent channels, **colour dimensions** can be either scalar or discrete [5]. Discrete colour dimensions follow a colour key where each colour represents a discrete value. Scalar colour dimensions follow a spectrum of colours used to represent a continuous range of values.
For many visualisations supported by this application and discussed in [5], the mandatory condition is that one or more of the chosen columns are have scalar data types. These conditions apply mostly to attributes selected by the user but can also include keys (such as for line charts). For a few visualisations, the conditions are stricter, focusing more on the exact data type rather than whether it is scalar or discrete. Examples of this are calendar charts requiring a temporal scalar attribute, word clouds requiring the key to be of a lexical data type, and choropleth maps requiring the key to be geographical. For the first two examples, we approach these rules by grouping PostgreSQL data types as below.

Lexical Data Types:

- VARCHAR
- TEXT
- CHAR
- BPCHAR

Numeric Data Types:

- NUMERIC
- INT-n where n is 2, 4, or 8
- FLOAT-n where n is 4 or 8

Temporal Data Types:

- DATE
- TIME
- TIMESTAMP

Note that the data types presented here are only those that we have encountered using Imperial's Mondial, Bank Branch and Family History databases. With this grouping, we can also identify which types represent scalar dimensions. In our implementation, we classify any data type within the numeric or temporal groups as scalar due to their natural ordering and continuous range of values.
Our final problem is with the choropleth map example. To determine whether a column has a geographical type requires looking beyond the data type alone. In our solution, we also look at both the column name and its table. If the column's name is geographical like `country`, `city`, `province` etc., we classify it as having a geographical type. In addition, if the table's name is geographical like `country`, `city`, `province` etc., and the column is `name`, `code`, or `id`, we classify it as having a geographical type. As with the data types, the geographical names of columns and tables considered are based on the databases used throughout this project.
We now use these rules regarding which data types are considered scalar, as well as the group each data type belongs to, in our decision making for the second round of pattern matching.

3.4 Generating Visualisations

In this section, we discuss the second stage of the visualisation recommendation process. We have a matched visualisation schema pattern and thus know the corresponding group of chart types to look through. Using our classification of the PostgreSQL data types from Section 3.3, we now look at which specific visualisations to recommend to the user based on their choice of columns. As previously stated, this round of pattern matching focuses on the chosen columns' data types as well as the number of attributes chosen by the user.

Key Cardinalities

Something we also propose is key cardinality limits for each visualisation type i.e. how much data we can visualise on one graph. The values chosen by myself, with the aid of McBrien's proposed key cardinalities in [5], are arbitrary, subjective, and justified by factors such as aesthetics and performance based on the limitations of the charting library used in this project. With the use of amCharts for most visualisations implemented in VizER, we found that some visualisation types could not handle large amounts of data well. Aesthetics-wise, some visualisation types like chord diagrams would not render properly if too much data was given. In terms of performance, trying to visualise too much data could cause the visualisation to lag, or in worst cases, freeze and crash.

Basic Visualisations

Since the pattern is a basic entity, we know that there are no foreign keys involved, only a single primary key. Recall that a basic entity is represented as a single entity E with key k and one or more attributes a_i. This is a simple case and as a result, there are many visualisations which fit into this category.

- **Bar Chart:** Each instance of the entity E is represented by a bar, with the length of each bar determined by a scalar attribute a_1.
- **Calendar Chart:** Each entry in the calendar represents an instance of the entity E according to a temporal scalar attribute a_1.
- **Scatter Chart:** Each point on the scatter chart represents an instance of the entity E, with scalar attributes a_1 and a_2 defining its coordinates,
- **Bubble Chart:** Similar to scatter chart, we have bubbles representing instances of E with scalar attributes a_1 and a_2 used to plot coordinates, and a third scalar attribute a_3 determining the bubble's size.
- **Choropleth Map:** The key column k represents some type of geographical region (continent, country, city etc.), of which each region represents an instance of the entity E, with its colour defined by the attribute a_1.
- **Word Cloud:** Each instance of the entity E is represented by a word (which has the value of the key column k), with the size of each word determined by scalar attribute a_1.

Chart Name	Key k Cardinality	Conditions
Bar Chart	1...100	a_1 scalar
Calendar Chart	1...*	a_1 temporal scalar
Scatter Chart	1...*	a_1, a_2 scalar
Bubble Chart	1...*	a_1, a_2, a_3 scalar
Choropleth Map	1...*	k geographical, a_1 colour
Word Cloud	1...*	k lexical, a_1 scalar

Table 3.1: Summary of basic visualisations, key cardinality constraints and conditions

Weak Visualisations

As discussed in Section 3.2, weak entities contain a compound key consisting of a foreign key k_1 to a parent entity E_p and a primary key belonging only to the child entity E_c. The corresponding visualisations involve grouping the data by the foreign key k_1 where each group has its own set of data identified by the primary key k_2. Some weak entity visualisations require the data to have an extra property of completeness. A weak entity is considered **complete** if for every k_1, there is the same set of values for k_2 [5]. An example of an incomplete weak entity is `country_population` (see Figure 3.5 - all countries do not have population figures for the same set of years). Therefore, as well as keeping track of the number of attributes chosen and the column data types, we sometimes need to ensure the weak entity is complete.

- **Line Chart:** Each line represents an instance of k_1. k_2 and an attribute a_1 are both required to be scalar as they represent the x and y-axis respectively.

- **Stacked Bar Chart:** The foreign key k_1 is shown as a bar (more specifically, a stack of bars), with each element within the stack representing an instance of k_2, and a scalar attribute a_1 governing the length of each element within the stack. This visualisation requires the completeness property from the weak entity.
- **Grouped Bar Chart:** Similar to a stacked bar chart, each group represents an instance of k_1 and each bar within a group is an instance of k_2. The length of each bar is determined by a scalar attribute a_1.
- **Spider Chart:** Each ring within the spider chart represents an instance of the foreign key k_1, and each spoke is an instance of k_2. The intersection of each ring with a spoke is determined by the scalar attribute a_1. This visualisation requires the completeness property from the weak entity.

Chart Name	Parent k_1 Card.	Child k_2 Card.	Completeness?	Conditions
Line Chart	1...20	1...*	No	k_2, a_1 scalar
Stacked Bar Chart	1...20	1...20	Yes	a_1 scalar
Scatter Chart	1...20	1...20	No	a_1 scalar
Spider Chart	3...10	1...20	Yes	a_1 scalar

Table 3.2: Summary of weak visualisations, key cardinality constraints and conditions

One-Many Visualisations

McBrien describes one-many visualisations as hierarchical in nature [5]. This is because, similar to weak entities, there is a parent-child relationship between the entity on the many side of the relationship (E_p) and the entity on the one side (E_c) respectively. However, unlike weak entities, the foreign key column is not part of the primary key. Each child key k_2 value is associated to one foreign k_1 value, enforcing the one-many cardinality. This changes our key cardinality constraints to consider the cardinality of k_2 per each k_1. The possible visualisations for this pattern are less common but are still interesting and meaningful.

- **Treemap:** Larger rectangles representing instances of E_p are divided into smaller rectangles representing instances of E_c; the size of each smaller rectangle is determined by scalar attribute a_1.
- **Hierarchy Tree:** Each node represents an instance of E_p connected by lines to circles (referred to as leaf nodes in the context of a tree) which represent instances of E_c.
- **Circle Packing:** Each circle represents an instance of E_p, with smaller circles attached to it representing instances of E_c, Depending on the charting library used, the smaller circles can be inside the parent circle or attached to its edge (our case in VizER is the latter). The scalar attribute a_1 determines the size of each smaller circle.

Chart Name	Parent k_1 Card.	Child k_2 Card. per k_1	Conditions
Treemap	1...20	1...20	a_1 scalar
Hierarchy Tree	1...20	1...20	-
Circe Packing	1...20	1...20	a_1 scalar

Table 3.3: Summary of one-many visualisations, key cardinality constraints and conditions

Many-Many Visualisations

Due to the non-restrictive cardinality for this type of relationship, network-related visualisations are the best fit for this schema pattern [5]. Each row in the joint table R represents a link between E_1 and E_2. The defining factor between recommending any of the following chart types is the nature of the many-many relationship. If the relationship is reflexive, a chord diagram is best suited due to its ability to display the interactions and relationships within an entity. Otherwise, a sankey diagram or network chart is recommended.

- **Sankey Diagram:** The elements on the left hand side represent instances of E_1, and the right hand side contains instances of E_2. This visualisation consists of visible flows between elements on the left hand side and elements on the right - the width of each flow is controlled by a scalar attribute a_1.
- **Chord Diagram:** Each instance of the entities is shown as a point on the circumference of a circle. Similar to sankey, the diagram consists of flows between points on the circle's edge, with the width of each flow determined by a scalar attribute a_1.
- **Network Chart:** Each instance of E_1 is represented by a larger circle, with the surrounding smaller circles representing instances of E_2. Links can be made between smaller circles and other circles if a connection exists in the data. The specific type of network chart used in VizER is the **Force-directed Network**.

Chart Name	k_1 Card.	Child k_2 Card.	Reflexive?	Conditions
Sankey Diagram	1...20	1...20	No	a_1 scalar
Chord Diagram	1...20	1...20	Yes	a_1 scalar
Network Chart	1...20	1...20	No	-

Table 3.4: Summary of many-many visualisations, key cardinality constraints and conditions

3.5 High-level Design and User Journeys

In this section, we apply the knowledge gained from Sections 3.2, 3.3 and 3.4 to our data visualiser. We utilise the predefined mappings between each schema pattern and their group of visualisations. We also adopt our key column analysis method from Section 3.2 to create mappings from the relational schema to the conceptual ER model. These mappings are used in two ways. In the DATA-FIRST mode, the user chooses a set of columns and we use our mappings on the user's chosen columns to recommend a set of visualisation types. In the VIZ-FIRST mode, we reverse this process and allow the user to choose a visualisation type, to which we use our mappings to identify a set of suitable tables, and column combinations which make suitable visualisation options.

3.5.1 Main Mode - DATA-FIRST

Figure 3.12 gives a high-level insight as to how the main mode of the application should run. Between the dotted lines, we have the user flow representing what the user would see and do on their screen. On the right, we have various backend operations. As shown in the diagram, there are three main stages on the user's side which lead to them viewing their requested visualisation.

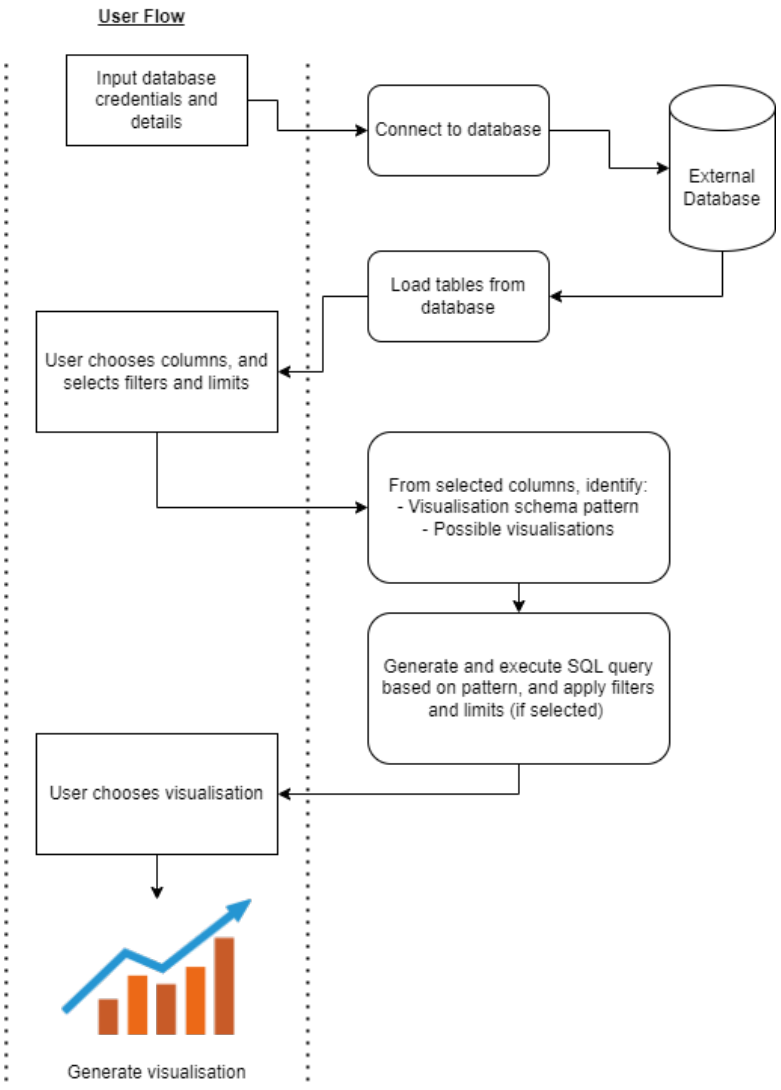


Figure 3.12: High-level design of the DATA-FIRST mode

Stage 1: Connecting to the database

As soon as the user loads the application, they will be faced with a form as shown in Gallery, Figure A.3. The specific details we need from the user to create the database connection are a username, password, host, port, and database name. The user's inputs are then sent to the backend to establish a connection to the user's database. After a connection is successfully created, all tables and their columns in the database are retrieved and sent to the frontend to be viewed by the user (see Gallery, Figure A.4). At the moment, only databases that use PostgreSQL DBMS are supported by the application.

Stage 2: User chooses columns they want to visualise

From the list of tables and their columns, the user chooses the data they wish to visualise by ticking checkboxes (Figure 3.13, Item 1). The user's selection of columns is sent to the backend and analysed to identify any matching schema pattern and all possible visualisations of this data. We decided on this approach, as opposed to reverse translating the entire relational schema upon connection to the database, in the interest of scalability and efficiency.
We discussed in Section 3.2, 3.3 and 3.4 the specific rules we use to identify the schema patterns and possible visualisation types. It is also important to note that the user's selection of columns will not always match a visualisation schema pattern. This "No Pattern Identified" result is still shown to the user so they understand this subset of data cannot be visualised by this system.
Once the pattern is identified and possible visualisation types are determined, the final step of this stage is generating the result data. An SQL query is built in the backend based on the schema pattern and the user's chosen columns - this query is then executed and the result data is sent back to view as a result table on the frontend. The user is also able to apply filters on columns (Figure 3.13, Item 2) as well as a limit on the number of rows returned (Figure 3.13, Item 3). For the filters, numeric columns can be filtered by the following operators: `=, !=, <, <=, >, >=`, and lexical columns can be filtered by the following string operators: `=, !=, LIKE, NOT LIKE`. These filters translate to `WHERE` clauses to be added to the SQL query. The limit input field is translated to a `LIMIT n` clause to be added to the SQL query.

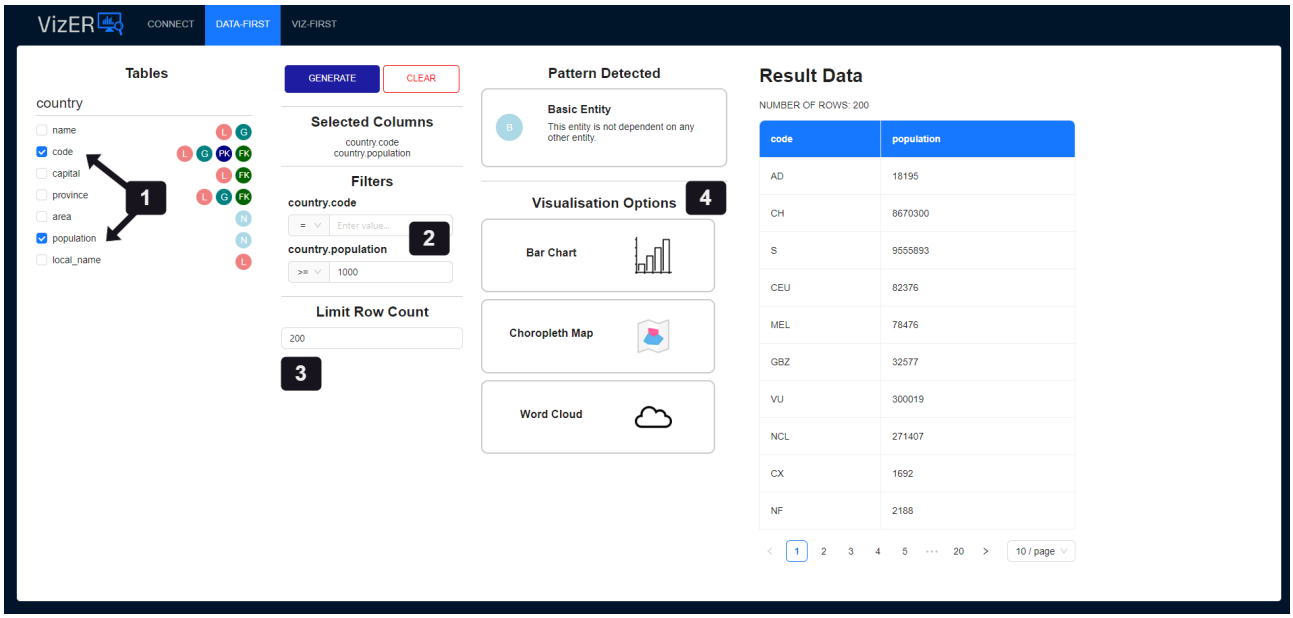


Figure 3.13: Visualising country populations

Stage 3: Generating the visualisation

A list of possible visualisation types is shown to the user, as shown in Figure 3.13, Item 4. The user clicks on a chart type to generate the visualisation. For some visualisations, the user can order the data to their interest (Figure 3.14, Item 1). There is also the option to truncate the data to follow the cardinality limits proposed by myself and [5] (Figure 3.14, Item 2) in Section 3.4. The ordering and truncation functionalities can be used together to identify interesting data trends and create more aesthetic graphs (in case the chart is too populated with data). All visualisations in this application are interactive - scroll bars, zooming/panning, and tool tip text all provide the user with an interactive experience when navigating the visualisation.

Figure 3.14: Bar chart visualisation of country populations

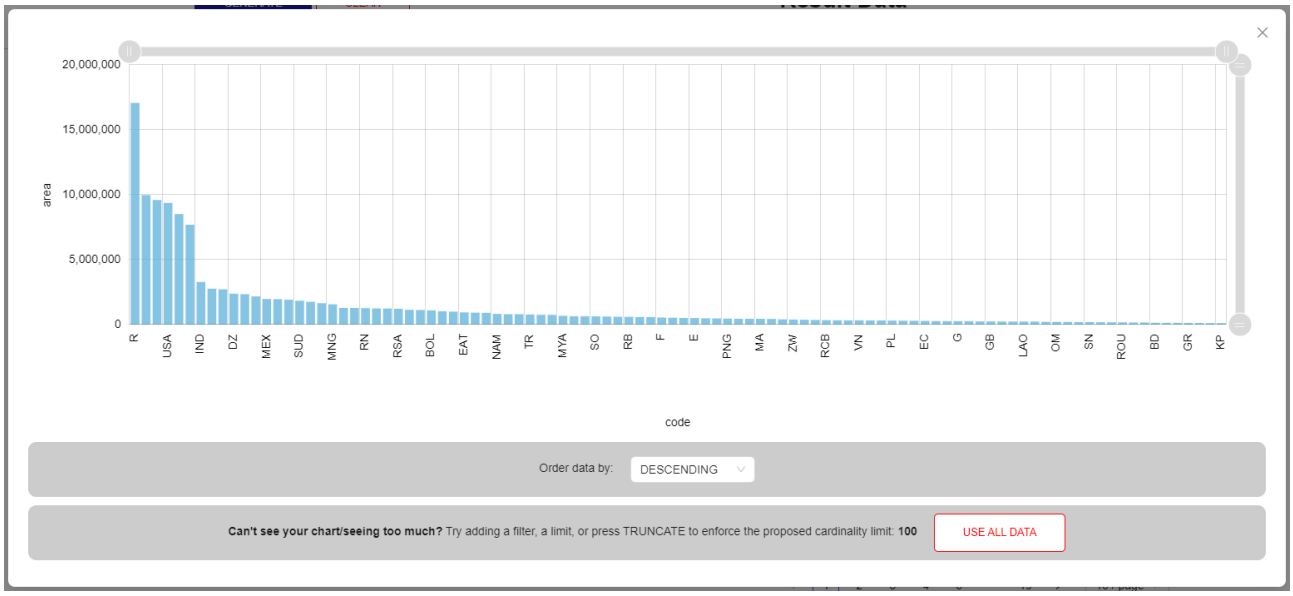


Figure 3.15: Bar chart visualisation after ordering and truncation

3.5.2 Extension - VIZ-FIRST

Figure 3.16 displays how the VIZ-FIRST mode of the application should run. This time, there are four stages for the user to generate a visualisation of their data. Note that this mode is an exploratory mode where the user will not have a specific idea on what data they wish to visualise.

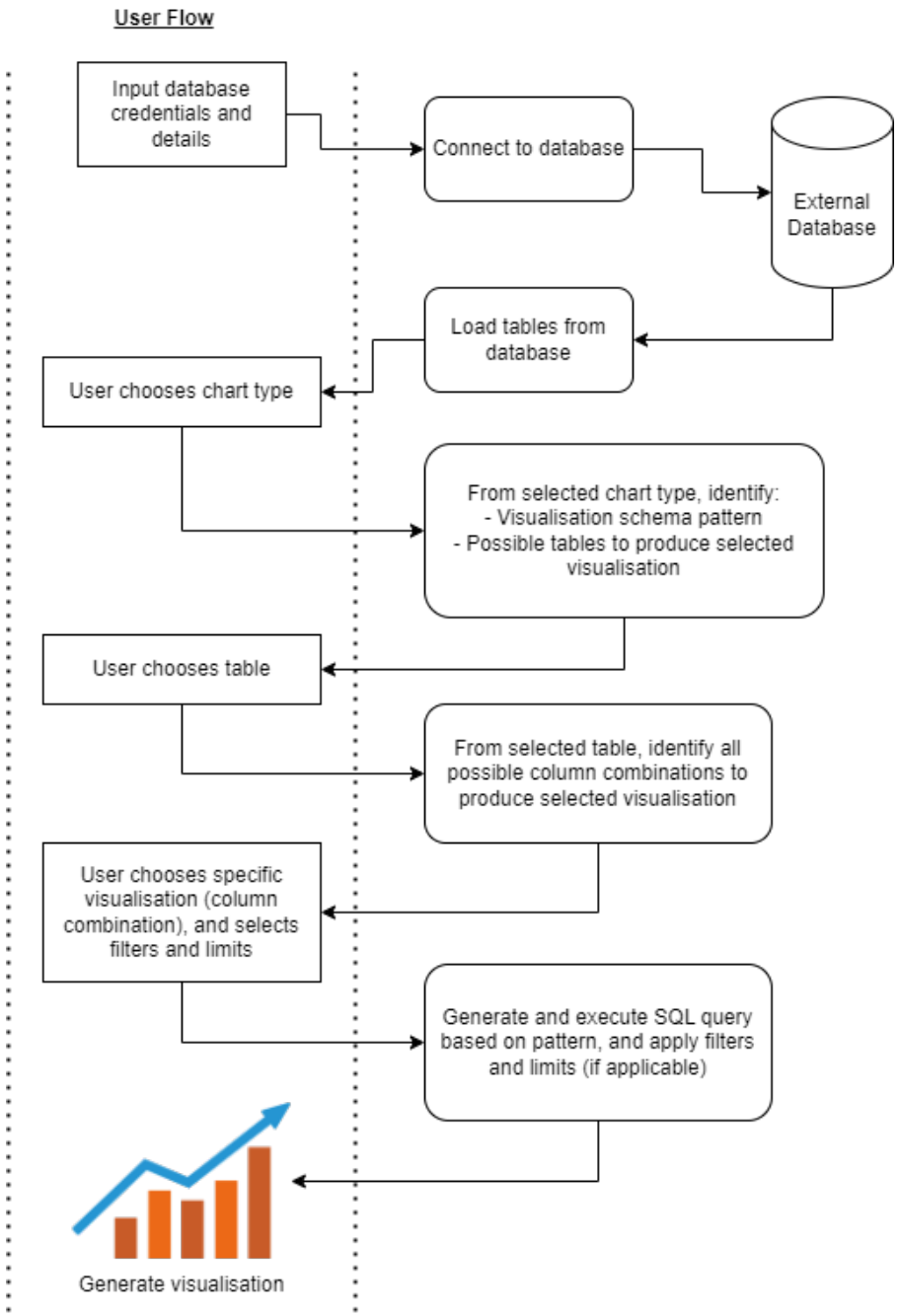


Figure 3.16: High-level design of the VIZ-FIRST mode

Stage 1: Connecting to the database

This is the same process as described in Section 3.5.1.

Stage 2: User chooses a chart type

Once the database connection is established, the user is shown a page similar to Gallery, Figure A.5. The user starts by navigating the list of visualisation types on the left hand side. Once the user chooses a chart type, this selection is sent to the backend. We find the corresponding visualisation schema pattern to the selected chart type. Once found, we only allow the user to select tables that map to this schema pattern - this prevents the user from attempting to create a chart with incompatible data. We return the filtered list of tables to the user (Figure 3.17).

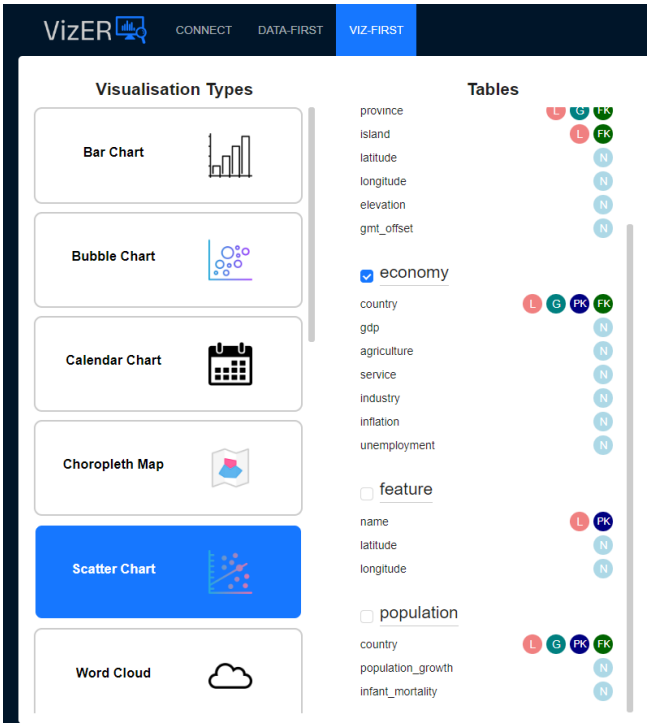


Figure 3.17: Filtered list of tables after choosing scatter graph

Stage 3: User chooses a table

The user chooses a table from the list shown. Once a table is chosen, we take in the type of chart we aim to produce and identify all possible column combinations from this table that can produce this chart type - this is done by analysing the data type of each column in this table. For example, if the user selected a bar chart, we return every possible pairing of the primary key column and a scalar attribute. For a simple chart type like scatter graphs and a table with many columns like `economy`, the backend returns an extensive list of all possible scatter graphs that can be created from the `economy` table (Figure 3.18(a)).

(a) (b)

Figure 3.18: (a) shows the result list of scatter charts, (b) shows the filtered list of scatter charts after setting the x-axis as `gdp`

Stage 4: User chooses a particular visualisation

The user navigates the list of possible visualisations and chooses a particular column combination. The user can set specific axes/fields to refine the search process, like only including visualisations with a particular column on the y-axis. This is shown in Figure 3.18(b) where we want scatter graphs for the `economy` table with gdp as the x-axis field. Similar to the DATA-FIRST mode, filters and row count limits can also be applied to the data to produce meaningful visualisations.

Stage 5: Generating the visualisation

When the user selects a particular visualisation (column combination), this selection is sent to the backend and the result data is generated, similarly as described in Section 3.5.1. Once the result data is sent to the frontend, the visualisation is shown to the user.

VizER: Implementation

Now that we have gone over the high-level approach, our next step is to build the application which follows the design shown in Figure 3.12 and 3.16. The project will be built as a full-stack application. The frontend will be responsible for creating the interface to facilitate the user's inputs and results. The backend will take in the user's inputs and apply the methods discussed in Section 3 to provide results without too much manual user input. In this way, the user is not required to have much technical knowledge nor do they need to use much manual effort to visualise their requested data.

4.1 Technical Choices

When it comes to what languages, frameworks, and tools to use to build this application, there are many possible routes to explore due to a vast range of resources available. In this section, the key criteria we use when discussing the technical decisions are:

- How well the application can meet the requirements discussed in Section 3.1.1 if built in this way.
- How accessible the resulting application will be if built in this way and what it would require from the user in order to run to its full capacity.
- My own skills and experiences, specifically how much building the application in this way would affect the project timeline (due to unfamiliarity with certain tools, languages etc.).

4.1.1 Application Type

A key consideration to make is what type the application should be: **desktop** or **web**. The final decision was to make VizER a web application due to the considerations below:

Web

- If the app were to be deployed and updated regularly, users would require no manual effort on their side to be able to use the **latest version** of the application.
- There is much more **availability for visualisation libraries** for web applications, as well as libraries for creating more aesthetic web pages.
- The web application would not be platform-dependent, it can be used on any browser and does not require anything but an internet connection, making it the more **accessible option**.
- My own **experiences and skills** made web the preferred choice. I have significantly more experience working on web applications than desktop. Therefore, I can focus my

efforts on the accuracy and functionality of the implementation rather than learning new languages and frameworks to build a desktop app.

Desktop

- Due to running natively on the user's machine, a desktop application could have **faster performance**.

- Since desktop applications do not require internet connection to run, users would be able to have **offline access**. However, one design requirement is to establish a connection to the user's database. Therefore, while users might be able to run the application offline, they would not be able to connect to any remote databases without an internet connection.

- Desktop applications have **limitations in accessibility**. The application would be platform-dependent (a version would have to be released for Windows, MacOS, Linux etc.). In addition, the application would only be available on the user's device that they installed on - they would not be able to access the application from any other device without redownloading.

It seems clear that a web application was the best choice given my own experience with web development, the surplus of available resources and libraries, and the lack of requirements on the user to be able to run the application themselves (not platform-dependent, no need for user to download on their device etc.).

4.1.2 Languages and Frameworks

Now that we knew the application will be web-based, we needed to consider what languages and frameworks could be used for both front end and back end. By separating the two halves of the application, we had the freedom of choosing the best language for each half. The final choice made was to implement the front end using JavaScript with the React JS framework, and to implement the backend using Java with the Spring Boot framework. This decision was mainly motivated by my level of experience and skill with these languages, and the available libraries and APIs compatible with these languages that could achieve the goals of the project.

Frontend

JavaScript is a high-level programming language I used for web development on the frontend. Specifically, I used the **ReactJS** library to build the user interface as it provided a simple way to build large and modern-looking web applications. Due to my proficiency in working with ReactJS, I was able to streamline the development of the front end application and focus on becoming familiar with the amCharts library for generating visualisations. ReactJS provided many advantages throughout the development process such as the virtual DOM that allowed for fast and dynamic updating and rendering of the UI, and the reusablility of React components since each mode of operation (DATA-FIRST and VIZ-FIRST) required separate pages that used a lot of the same components. For the frontend, other alternative frameworks could have been used such as Angular and Vue.js, or alternative languages such as using Java with Spring Boot and the Vaadin framework for the frontend, but my proficiency in ReactJS and the large wealth of online resources made it the more preferable choice.

Backend

The language used for the backend of VizER is **Java**, an object-oriented programming language known for building performant and stable applications. The **Spring Boot** framework is used to reduce the complexity of setting up the Java application and facilitates the communication to the frontend via RESTful API endpoints. As the developer for this project, I did have a lot of experience working with Java and while I had no experience using Spring Boot, it seemed easy to learn and integrate and was designed to simplify the development of RESTful APIs. Spring Boot also has pre-configured project templates, including a JDBC starter template to allow quick and easy integration of the JDBC API in the backend. There were various alternative options for backend languages and frameworks. One option was to use Node.js with Express based on my past experience working on backend services with this framework, however Java had the advantage with the JDBC API which allowed easy interactivity with databases, such as accessing the database's metadata.

4.1.3 Libraries and APIs Used

There were a multitude of libraries and APIs used in both the front and backend of VizER, for a range of purposes. These tools provided support in both the functionality of the application and the aesthetics of the UI to enable a seamless and exciting user experience.

- **JDBC (Java Database Connectivity) API** was used in the backend to facilitate the interaction with the user's database. The API was used to not only connect to the database and execute SQL queries, but also to access the database's metadata. The metadata allowed us to examine each table's primary and foreign keys in detail as well as access the data types of each column, all used in the visualisation recommendation process.

- **Ant Design (AntD)** is a vast library of pre-built UI components for React. The library was full of high-quality interactive components that saved a lot of time during development and provided the user with a consistent look. The AntD components used were checkboxes for the user's column selection, inputs for both the database connection page and for the filters and row-count limits, selectors for the navigation process in the VIZ-FIRST mode, the result table in the DATA-FIRST mode used to display the user's selected data, and the modal which displays the user's selected visualisation.

- **amCharts** is a charting library with an extensive range of interactive graphs, providing both simple (e.g. bar chart, scatter graph) and complex (e.g. circle packing, chord diagram) data visualisations. The library is easier to implement and more well-documented compared to grammars discussed in Section 2.1.3, and more low-level libraries such as D3.js, but still retains a good level of control and customisation over the different components of each graph. The motivation behind choosing this charting library was their wide range of available charts (along with tutorials and documentation for each type).

- **Google Charts** is also a charting library which has a range of interactive charts for web applications. My main motivation for integrating this second charting library was because amCharts did not have any Calendar Chart options. Based on my own research, Google Charts provided simple integration and configuration of this chart type, therefore I chose it specifically for the calendar chart as it did not have a wide enough range to replace amCharts completely.

4.2 Architecture Diagram

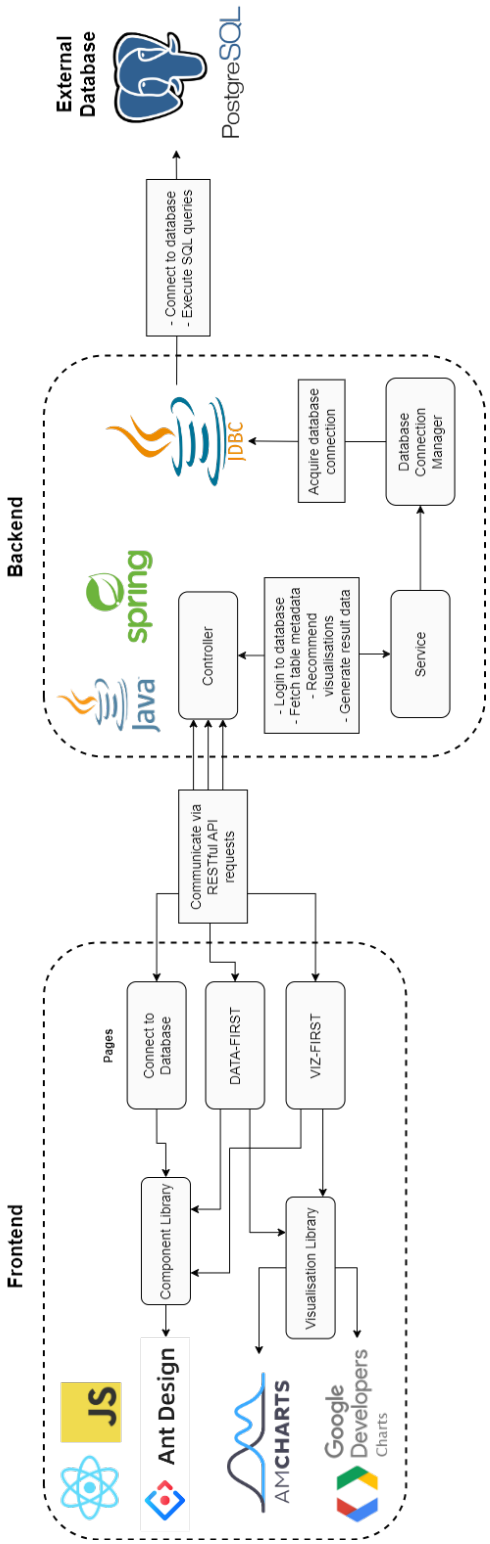


Figure 4.1: Architecture diagram for VizER

Figure 4.1 displays an architecture diagram for VizER, giving a greater in-depth view of what the application is composed of. The solid rounded boxes represent internal components within the application such as pages, classes, libraries etc. and the square boxes represent interactions between different components throughout the user journey.

4.3 API Sequence Diagrams

4.3.1 DATA-FIRST Mode

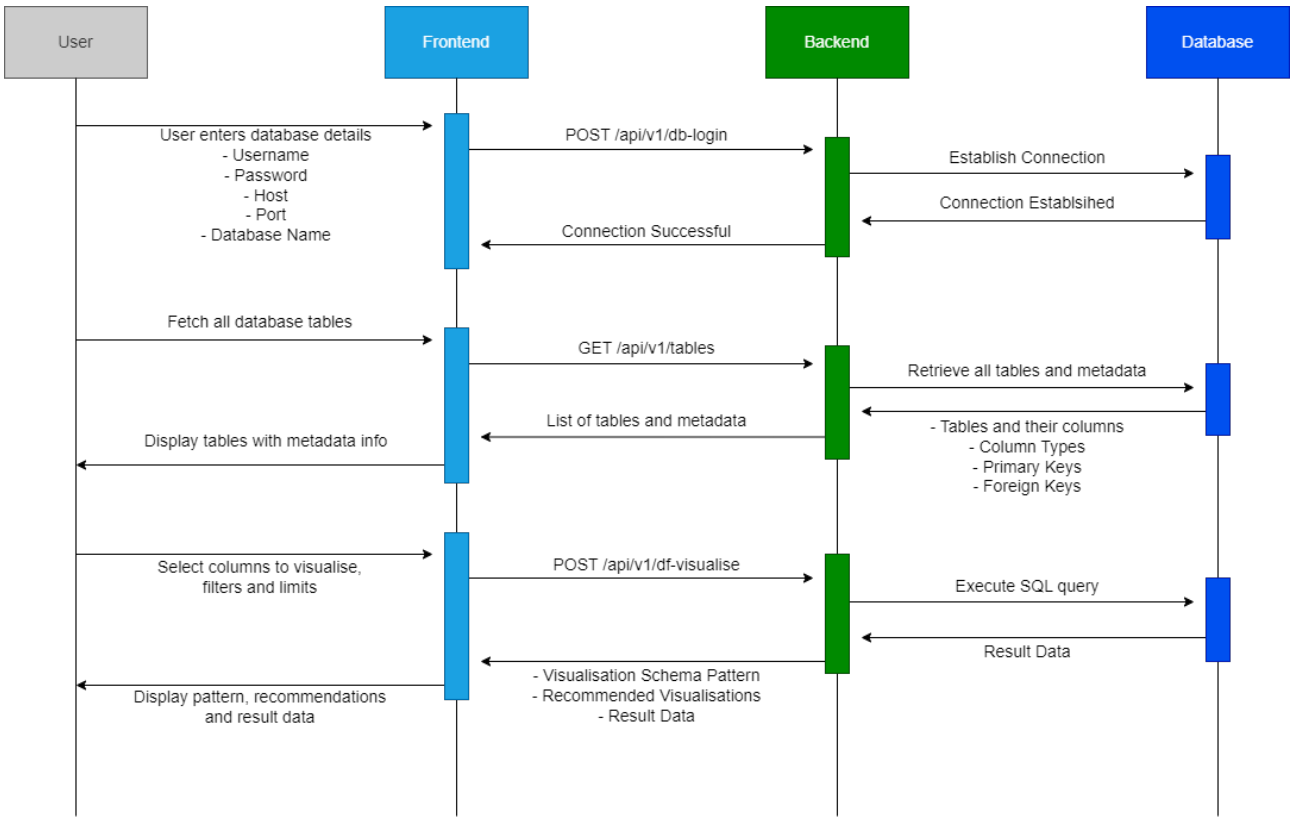


Figure 4.2: API Sequence diagram for DATA-FIRST mode

Figure 4.2 is an API sequence diagram for the main mode of operation, DATA-FIRST. This diagram shows all communications occurring between the frontend, backend and database; each communication is triggered by the user's actions. We see here exactly what API requests are being made to the backend, what responses are delivered back to the frontend and what the body of each request/response contains.

4.3.2 VIZ-FIRST Mode

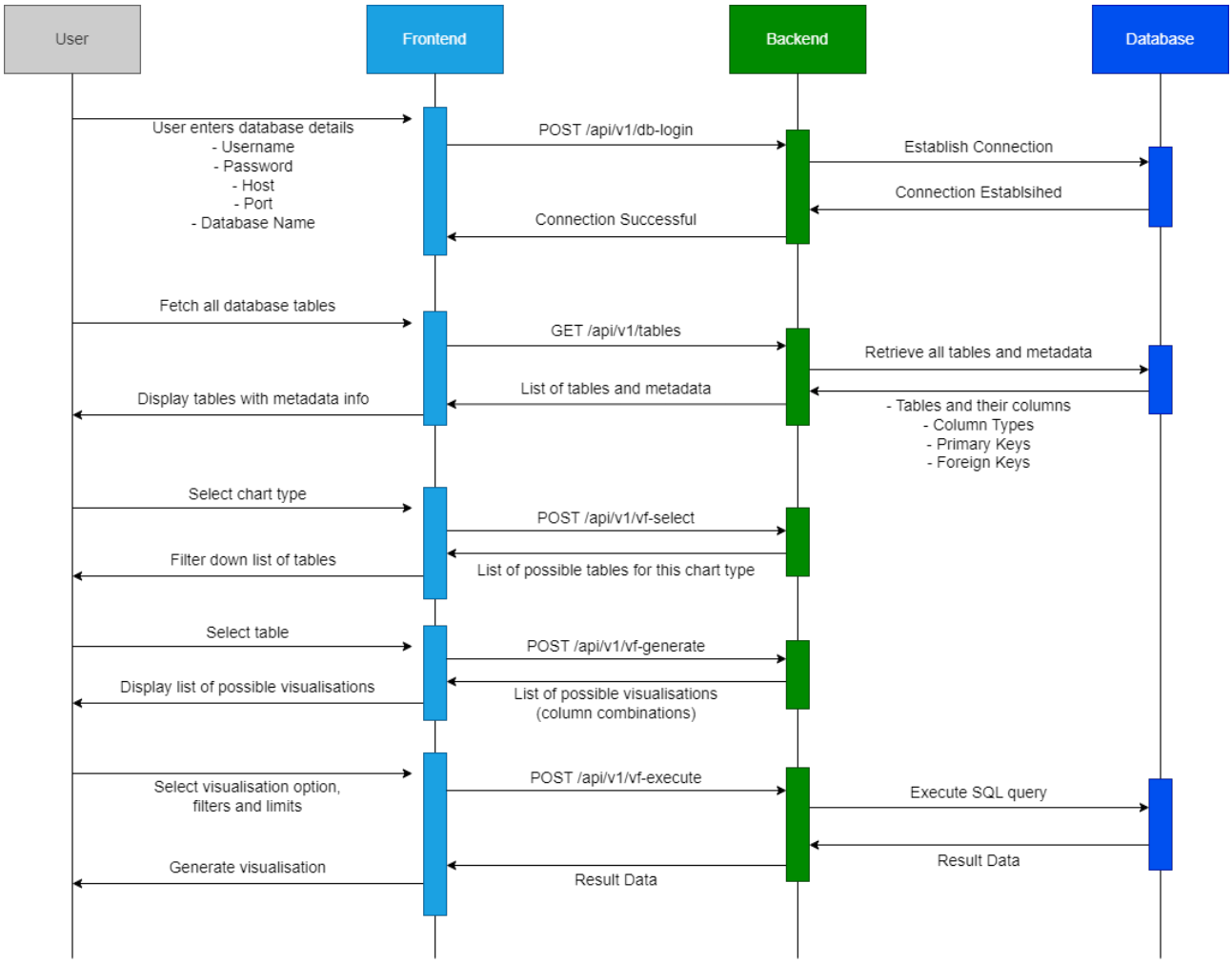


Figure 4.3: API Sequence diagram for VIZ-FIRST mode

Similarly, Figure 4.3 shows an API sequence diagram for the secondary mode of operation, the VIZ-FIRST mode, which is the extension for this project. Here, we send an API request to filter down the list of tables after the user selects a chart type, and another API request to generate all possible column combinations after the user selects the table they want to visualise data from. The final API request is sent after the user locks in their visualisation choice, so that we know what specific columns the user wishes to visualise and can execute the right SQL query to get the result data. This differs from the DATA-FIRST mode which has the opposite order of events - in DATA-FIRST, the user selects their columns (and table) first, and then their choice of visualisation.

4.4 Technical Implementation

In this section, we go through a more detailed description of the user journeys discussed in Section 3.2, explaining what is happening within the program at each step, and how this achieves the goals outlined in Section 3.1.1.

4.4.1 DATA-FIRST Mode

Stage 1: Connecting to the database

As stated previously, the first page the user will see once they load the application is a `LoginPage` where the user will need to give the necessary credentials for the system to connect to their database. The form for the user's database details is shown properly in Figure 4.4 where the following details are requested:

- **Username:** their username for logging into the database.
- **Password:** their password for logging into the database.
- **Host:** the URL of the server hosting the user's database.
- **Port:** the port of which the system can communicate with the database.
- **Database Name:** the name of the specific database they want to use.

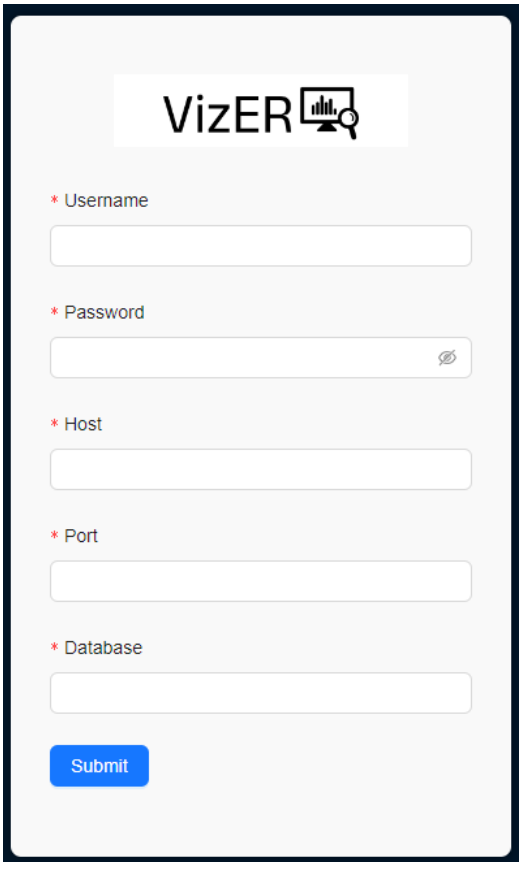


Figure 4.4: Form component for the `LoginPage` page

Once the user presses the submit button, we send a POST request to the backend containing the form's data in JSON format below:

```
{
    username: <username>,
    password: <password>,
    host: <host URL>,
    port: <port number>,
    database: <database name>
}
```

The Java Spring backend receives this API request at the endpoint `/api/v1/db-login`, with the request body set as a `DatabaseDetails` object. The `MainController` passes the body into the `setConnectionDetails` function of the `DatabaseService` class. This function uses a `DatabaseConnectionManager` object (a field of `DatabaseService`) to establish a connection with the database. This is done by creating a connection URL of the following format:

```
jdbc:postgresql//<host>:<port>/<database>
```

The JDBC API and the JDBC PostgreSQL driver are used to create a connection to the user's Postgres database. At the moment, only this DBMS is supported by the VizER application. Once a connection to the database is successfully established, an API response with an OK status is sent back to the frontend, else a response with a `500 Server Error` is sent instead. The page displays a confirmation popup message regarding whether the connection was successful or not. In the case of an unsuccessful connection, the user can adjust their entries in the form and resubmit - an unsuccessful connection does not crash the backend.
After a successful database connection, the application redirects to the DATA-FIRST page. Once the page loads, a GET request is instantly sent to the backend's `/api/v1/tables` endpoint in order to retrieve all the user's database tables and their metadata.
The `fetchTableMetadata` function in the `DatabaseService` uses the JDBC API to retrieve the database's metadata. From this, we use the `getTables` method of the `DatabaseMetadata` class to extract all the tables from the public schema, ensuring to use the "TABLES" keyword to exclude views. We loop through every table to extract its name, columns' names, columns' types, primary key names, and foreign keys. Each column name and type is saved as a `Column` object along with the table name and boolean fields `isPrimaryKey` and `isForeignKey`. Each foreign key is saved as a `ForeignKey` object with extra information about its parent table and parent column name (e.g. `country1` column in `borders` table is a foreign key to the `code` column of the `country` table, see Figure 3.11). Finally, a `TableMetadata` object is created for each table which has the following definition:

```
public class TableMetadata {
    private final String tableName;
    private final List<Column> columns;
    private final List<String> primaryKeys;
    private final List<ForeignKey> foreignKeys;
}
```

This list of `TableMetadata` objects is saved as the `databaseMetadata` field for the `DatabaseService` class and also sent as the body of the API response back to the frontend. This is then displayed on the user's screen as a list of tables and columns. There are information tags next to each column regarding the column's type (lexical, numerical, temporal or geographical), and whether it is a primary and/or foreign key.

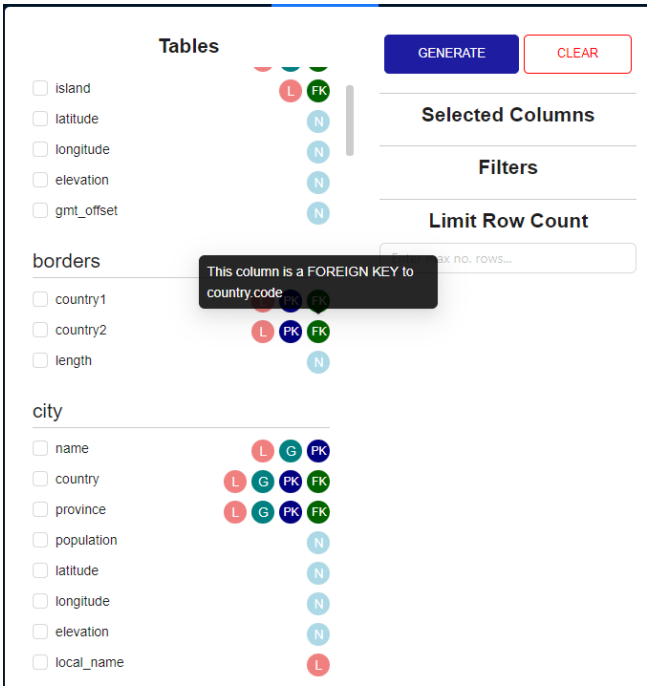


Figure 4.5: List of tables and their metadata. Upon hovering over the FK tag for `country2` of the `borders` table, we have more information about this foreign key.

Stage 2: Choosing columns to visualise

The list of tables have a checkbox next to each column. This allows the user to select the exact columns they wish to visualise. For each of their selected columns, they are able to apply filters on the result data for more interesting visualisations. Numeric columns are filtered using the `=, !=, <, <=, >, >=` operators. Lexical columns are filtered using the `=, !=` and the SQL operators `LIKE, NOT LIKE`. The filter value is an input by the user. The user can also limit the row count of the result data. By doing this, the user can handle tables with large volumes of data.

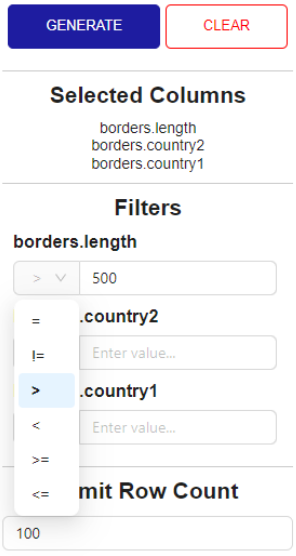


Figure 4.6: The filter and limit components of the DATA-FIRST page. For the filters, the users select the type of constraint they want using the selector on the left and input the constraint value in the input box. Here, they have filtered `borders.length` to be > 500 and they have limited the row count to 100 rows.

The user can clear their selection of columns using the CLEAR button shown in Figure 4.6. Once the user is finished with selection, they press the GENERATE button which sends a POST request to the backend endpoint `/api/v1/df-visualise`. The body of this is a JSON object of format:

```
{
    tables: <list of selected tables>,
    columns: <list of full column names>,
    filters: <filters that are being applied>,
    <limit>: <limit value>
}
```

By full column names, we refer to the column name with its table as a prefix such as `borders.length`. The filters field is an object acting as a map where the keys are the names of columns being filtered and the values are the corresponding filter objects. A filter object has the following format:

```
{
    type: <"num" if numeric filter else "lex">,
    comparator: <filter operator e.g. "=">,
    value: <filter value>
}
```

Additionally, if the limit input does not have a value (i.e. the user does not want to impose a limit on the row count), a value of -1 is sent instead. Once the backend receives this POST request, it passes the body (saved as a `DFRequest` object) into the `dfReccomendVisualisations` function of the `DatabaseService`. This function performs both rounds of pattern matching following the rules discussed in Sections 3.3, 3.4, and 3.5. After identifying the pattern using functions such as `isBasicEntity` to determine whether the selection of columns matches that ER pattern, we use functions such as `bar`, `scatter`, and `wordCloud` to determine whether the chosen columns are suitable for that specific visualisation type. These functions look like below, analysing the number of attributes chosen and the types of the columns chosen (whether that be scalar/discrete or for a particular group like lexical):

```
private boolean bar(List<String> attTypes) {
  return attTypes.size() == 1 && isScalarType(attTypes.get(0));
}

private boolean wordCloud(Column key, List<String> attTypes) {
  return attTypes.size() == 1 && LEX_TYPES.contains(key.getType())
      && isScalarType(attTypes.get(0));
}

private boolean scatter(List<String> attTypes) {
  return attTypes.size() == 2
      && attTypes.stream().allMatch(this::isScalarType);
}
```

If the user's selection is identified as matching a weak entity, there is an extra step involved when checking for a stacked bar chart or spider chart: **completeness**. We check if a weak entity is complete by running an SQL query for the key columns on the chosen table, specifically checking that for every parent key column value, there exists the same set of child key column values. A `VisualisationOption` object is created for every suitable chart type based on the chosen columns.

Once a pattern is identified and the list of `VisualisationOption` objects is created, the final step is to generate the result data. Depending on the schema pattern identified, we run a specific SQL query using JDBC to get the result data. For example, the SQL query for a weak entity will require us to group the data by the parent key column in order to aggregate the information for each (k_1, k_2) combination - we use SUM for aggregation. We ensure that for compound foreign keys, we concatenate the values as the charting library only allows one column in the data object per axis/field. To ensure every record of data can be visualised, we do not allow any null values as this would cause error upon visualisation (the mandatory attribute specialisation transformation [5]). For filters, we translate each filter object into part of the WHERE clause. Any limit value is concatenated to the SQL statement at the end as a `LIMIT n` clause. An example query for the weak entity `city` is shown below:

```
SELECT country || " | " || province AS "country | province",
    name,
    SUM(population) AS population
FROM city
WHERE country IS NOT NULL
    AND province IS NOT NULL
    AND city IS NOT NULL
    AND population IS NOT NULL
    AND population >= 5000
GROUP BY city, country, province
ORDER BY city, country, province
LIMIT 100;
```

The pattern, list of visualisation options, and result data are all sent back to the frontend as a `DFResponse` object. Note that each visualisation option is an object of the following format:

```
public class VisualisationOption {
    private final String id;
    private final String name;
    private final String key1;
    private final String key2;
    private final List<String> attributes;
    private final String title;
}
```

The user will see the identified schema pattern, the list of visualisations they can generate, and the result table on their screen.

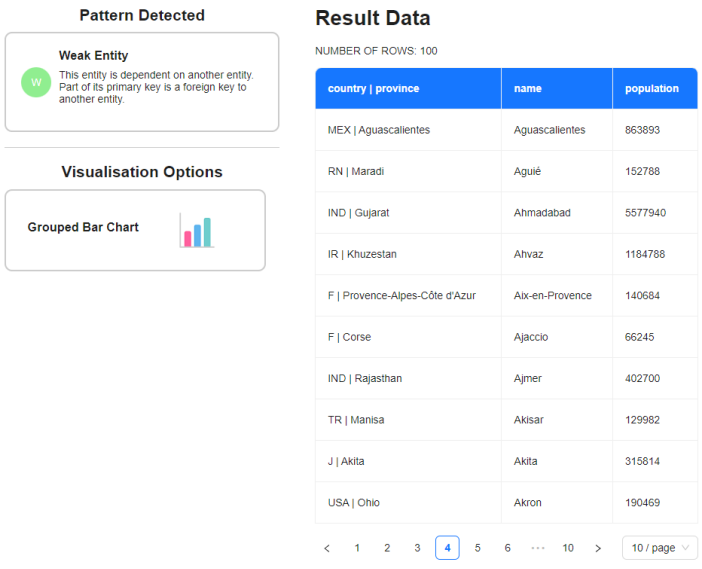
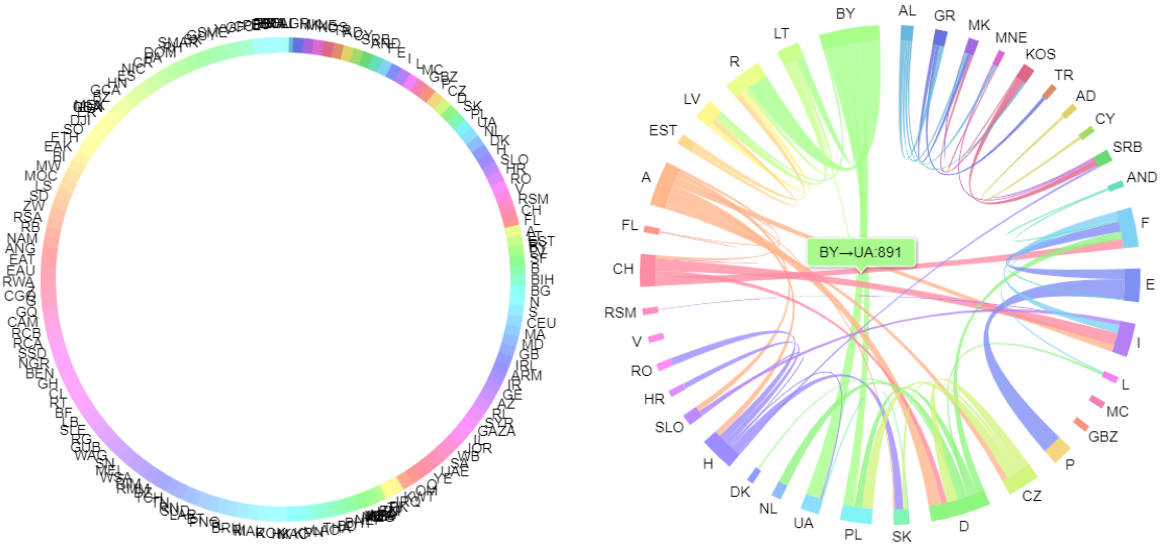


Figure 4.7: Results of the example query from Stage 2

Stage 3: Generating the visualisation

This stage requires no communication with the backend. The user goes through the list of recommended visualisations and clicks the type they wish to generate. This triggers the `generateChart` function which sets the type of graph to be rendered. Each graph is its own separate React component, taking in the chart data and the fields it needs as parameters. By having each chart type as a separate React component, it is easier for the addition of new chart types in the future, allowing our solution to be extensible. All that needs to be done is creating the component for the new visualisation type and adding the option to `generateChart`. Also, any changes to a specific chart component will not affect any other charts. Once the graph is rendered, the user has two functionalities below the graph: ordering and truncation. The user is able to order the chart data in ascending or descending order. The TRUNCATE button's main purpose is to be used when the chart data is too large. This can result in a cluttered graph or in worse cases, a graph that will not render properly (an example of such is shown in Figure 4.8). Pressing the TRUNCATE button enforces the proposed cardinality constraints discussed in Section 3.5. In conjunction, the ordering and truncation functionalities can be used to create aesthetic and meaningful visualisations by focusing on trends in the data.



(a) No truncation

(b) With truncation

Figure 4.8: Visualising the borders between countries and their lengths

4.4.2 VIZ-FIRST Mode

Stage 1: Connecting to the database

This follows the same procedure as Section 4.4.1. Once the user is redirected to the DATA-FIRST page after connecting, they click the VIZ-FIRST button in the top navigation bar to access the VIZ-FIRST mode.

Stage 2: User chooses a chart type

The user navigates through the list of all visualisations and picks which of them they want to generate. Once a chart type is selected, a POST request is sent to the backend API endpoint `/api/v1/vf-select` with the ID of the chart type ("bar", "word-cloud", "grouped-bar" etc.) as the body. The `MainController` passes this chart ID to the `vfSelectVis` function of the `DatabaseService`. This function iterates through every `TableMetadata` object in the `databaseMetadata` field and checks:

- Whether the table can be represented as the chart type's corresponding schema pattern (e.g. pass the table into the `isManyManyRelationship` function if the chart ID is "chord", "sankey", or "network-chart").
- Whether there are columns within the table that can be visualised as the selected chart type (e.g. for a bubble chart, check that the table contains at least three columns with scalar data types).

The API response is a list of all tables appropriate for the selected visualisation type. The frontend will then filter down the shown list of tables to only include those that are in this list of appropriate tables thus can be visualised as the selected chart type.

Stage 3: User chooses a table

The user looks through the filtered list of tables that can be visualised as their selected chart type. Unlike the DATA-FIRST mode, this time the columns are not being selected but the tables themselves. This is because this mode is designed to help the user find column selections from a chosen table that can be visualised as their selected chart type. As before, the user can clear their selections using the CLEAR button. Once the GENERATE button is pressed, a POST request is sent to the backend with the chart ID and table name as the JSON body. Once the backend receives this at the `/api/v1/vf-generate` endpoint, the `MainController` passes the request body to `vfGenerateOptions` in the `DatabaseService`. This function iterates over every combination of attributes (and keys in the one-many case) in the selected table and collects a list of all possible visualisations that can be created of the chosen type. For every suitable column combination, a `VisualisationOption` object is created with its information, and added to a list of all possible visualisation options. An example of this is checking which column combinations from the `economy` table could be visualised as a scatter chart. In Figure 4.9, we have the `economy` table.

Economy	
PK FK	country (LEXICAL)
	gdp (NUMERIC)
	agriculture (NUMERIC)
	service (NUMERIC)
	industry (NUMERIC)
	inflation (NUMERIC)
	unemployment (NUMERIC)

Figure 4.9: Economy table

Recall that in our system, a scatter chart requires:

- A Basic Entity pattern
- Two scalar attributes

From the table, we see that there are no foreign key columns therefore this table maps to a basic entity. We also see that there are at least two attributes that have scalar data types (as they are numeric), therefore we know a scatter chart can be created from this table. By iterating over every pair of scalar attributes (order matters here - (a,b) is not the same as (b,a)), we see there are 30 possible column combinations. The `VFResponse` object sent back to the frontend contains the name of the schema pattern for the chosen chart type and a list of `VisualisationOption` objects. The screen now displays a list of visualisation options.

Stage 4: User chooses a visualisation

The user will see a screen similar to Figure 3.10 with a list of visualisation options on the far right and various options to refine the search. The user can filter the list of visualisation options by setting the axes to be a specific column. For every column that the user sets as a specific field, they can apply filters similar to the DATA-FIRST mode for the value of that column e.g. setting gdp as the x-axis for the scatter graph and filtering gdp to be greater than 10000. Similar to the DATA-FIRST mode, the user can also apply a limit to the row count.

Stage 5: Generating the visualisation

Once the user has decided on which specific visualisation option they want to view, the last step is to retrieve the result data for this visualisation. When the user clicks a visualisation option, a POST request is sent to the backend with the following JSON body:

```
{
    pattern: <schema pattern for this chart>,
    tables: <selected table name>,
    columns: <specific column combination to visualise>,
    filters: <filters that are being applied>,
    limit: <limit value>
}
```

This format is similar to the JSON body sent to retrieve the results for the DATA-FIRST mode with the addition of the `pattern` field. The reason for including this field is so that in the backend, we do not need to reidentify the schema pattern to determine which SQL query to execute. The API request is sent to the backend's `/api/v1/vf-execute` endpoint where it is

passed into the `vfExecuteQuery` function in the `DatabaseService`. This function builds and executes the SQL query corresponding to the schema pattern. The result data is sent back to the frontend and the visualisation is generated. Once the visualisation is displayed to the user, they will have the same ordering and truncation functionalities available as the DATA-FIRST mode.

4.4.3 Problems Encountered During Development

Throughout the development process, there were some setbacks I faced which caused me to rethink my options and purse alternative pathways:

- Calendar charts were not available in amCharts. For this reason, I pursued other charting libraries that could provide this visualisation. In the interest of time, I looked for libraries with simple calendar chart implementations. Upon finding Google Charts, I was able to easily integrate the calendar chart despite involving a different charting library due to the separation of visualisation types into their own React components.

- My initial approach to this project was to reverse translate the entire database in the back-end upon establishing a connection. However, this would have been a time-consuming task to implement, and I would have run the risk of losing time toward the end of the project, such as with the VIZ-FIRST extension. The Amazing-ER library, used by David Bull [6] in his solution (though not for this purpose), was an option I pursued. Unfortunately, the reverse translation process failed for weak entities thus I was unable to use this tool for my application. Instead, I went with a more efficient column analysis method, focusing on mapping the user's **selected data** to a schema pattern as opposed to every table in the database.

Evaluation

5.1 Core Functionality

We identify the following success criteria for this project and the VizER application.

Application Goals

- Connect to the user's database and display all tables and their metadata.
- **DATA-FIRST**
  - Identify the correct visualisation schema pattern from the user's selected data and the conceptual ER model of the data.
  - Identify the correct set of suitable visualisations based on the mappings between the schema patterns and groups of visualisations.
- **VIZ-FIRST**
  - Identify the correct set of appropriate tables based on the user's selected visualisation type and the conceptual ER model of the data.
  - Identify possible visualisation options by exploring all column configurations from the user's selected visualisation type and table.

Project Goals

- Create an application that is extensible and can be improved in the future to adopt new visualisation types and/or schema patterns.
- Create an application that is scalable and can handle both large databases and the visualisation of large volumes of data.
- Create an application that is easy-to-use and can be enjoyed by both technical and non-technical users.

5.1.1 Identification of visualisation schema patterns

Multiple tests were conducted across the Mondial, Bank Branch and Family History database for the correct identification of visualisation schema patterns from the user's selected data. We ensured all five schema patterns could be identified by the system and presented correctly to the user. These tests also checked the system could connect to different Postgres databases. Example tests are shown in Table 5.1. We included a second test for the basic entity when its primary key is completely inherited from a parent entity.

Table	Chosen Columns	Primary Keys	Foreign Keys	Identified Pattern	PASS/FAIL?
Country	code, population	code	-	Basic	PASS
Economy	country, gdp, unemployment	country	country	Basic	PASS
Ethnic Group	country, name, percentage	country, name	country	Weak	PASS
Airport	iata_code, island, elevation	iata_code	island	One-Many	PASS
Encompasses	country, continent, percentage	country, continent	country, continent	Many-Many	PASS
Borders	country1, country2, length	country1, country2	country1, country2	Reflexive Many-Many	PASS

Table 5.1: Example tests for identifying the correct visualisation schema pattern

We see all visualisation schema patterns can be identified correctly. However, we discovered a problem with weak entities. Recall that a weak entity's primary key consists of a parent primary key which is foreign, and a child primary key. If the user selects only the child primary key, the system classes this as a basic entity due to no foreign keys being present in the selected data. Thus, basic visualisations are recommended to the user which would contain duplicates due to an non-unique chosen primary key (Figure 5.1). To solve this problem, we can adjust the backend's `isWeakEntity` function to include the table's **entire** primary key in the column analysis instead of solely analysing the selected columns.

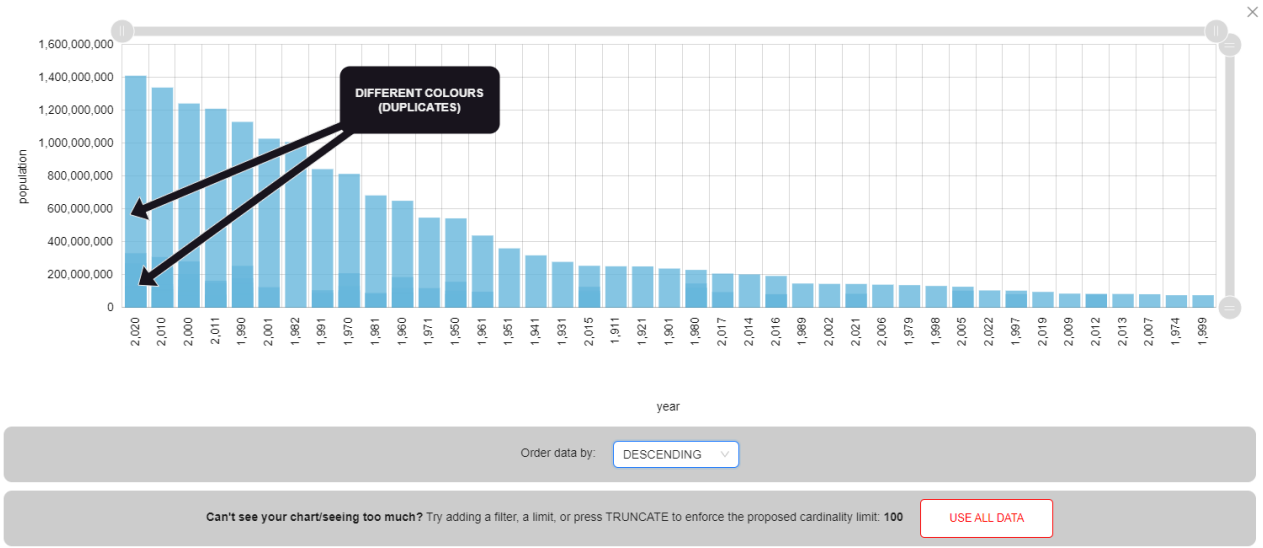


Figure 5.1: Bar chart with duplicate data due to an non-unique chosen primary key

5.1.2 Recommending suitable visualisations

We then tested the application's ability to perform the second part of the DATA-FIRST mode: using mappings between the schema patterns and visualisation types to recommend users the best visualisations for their selected data. Example tests are shown in Table 5.2, following the same examples as before. The data types are labelled as (N) Numeric, (L) Lexical, (T) Temporal, and (G) Geographical.

Table	Chosen Columns (Data Type)	Scalar Attributes	Recommendation Visualisation(s)	PASS/FAIL?
Country	code (L, G), population (N)	population	Bar Chart, Choropleth Map, Word Cloud	PASS
Economy	country (L, G), gdp (N), inflation (N)	gdp, inflation	Scatter Chart	PASS
Lake	name (L), elevation (N), depth (N), dam_height (N)	elevation, depth, dam_height	Bubble Chart	PASS
Organisation	abbreviation (L), established (T)	established	Calendar Chart, Bar Chart, Word Cloud	PASS
Country Population	country (L, G), year (T), population (N)	year, population	Line Chart	PASS
Ethnic Group	country (L, G), name (L), percentage (N)	percentage	Grouped Bar Chart	PASS
Airport	iata_code (L), island (L), elevation (N)	elevation	Treemap, Circle Packing	PASS
Encompasses	country (L, G), continent (L, G), percentage (N)	percentage	Sankey Diagram	PASS
Borders	country1 (L, G), country2 (L, G), length (N)	length	Chord Diagram, Network Chart	PASS

Table 5.2: Example tests for recommending suitable visualisations

5.1.3 VIZ-FIRST: Exploring all visualisation options

We moved onto testing the VIZ-FIRST mode. Here, we ensured that based on the user's selected visualisation type and table, the system produced a list of all possible column combinations, which could be refined down if the user set a particular axis/field. We used similar test cases to our DATA-FIRST tests to ensure VIZ-FIRST can handle all schema patterns and visualisation types. Example tests are shown in Table 5.3.

Visualisation Type	Table	Identified Pattern	Number of options	PASS/FAIL?
Bar Chart	Continent	Basic	1	PASS
Choropleth Map	Country	Basic	2	PASS
Scatter Chart	Economy	Basic	30	PASS
Bubble Chart	River	Basic	120	PASS
Line Chart	Country Development	Weak	5	PASS
Grouped Bar Chart	City	Weak	4	PASS
Treemap	Country	One-Many	2	PASS
Sankey Diagram	Encompasses	Many-Many	1	PASS
Chord Diagram	Borders	Reflexive Many-Many	1	PASS

Table 5.3: Example tests for generating visualisation options in VIZ-FIRST mode

5.1.4 Scalability and Extensibility

Connecting to and using large databases

Throughout the development and testing process, we connected VizER to multiple databases of various sizes. Our largest was the Mondial database with 39 tables, with the largest table containing 12148 records. The application loaded this database and processed its data in both modes with no issues. This is partially due to our approach of schema pattern identification. By only translating the user's chosen columns to ER representation, as opposed to reverse translating the entire relational schema upon connecting to the database, we lessen the load on the backend and reduce any redundant processing and translating of unused tables.

Visualising high volumes of data

We evaluated VizER's ability to visualise large volumes of data. As discussed in Section 3.4, the amCharts library suffers performance issues when visualising high volumes of data - the chart will lag or in extreme cases, the application will freeze and crash. This "extreme case" only happens when visualising substantial volumes of data (e.g. the 12148 rows of the City Population table) in an intricate format like line charts. We deal with this extreme case using cardinality limits. For some charts, a cardinality limit is automatically enforced on the chart data to maintain the the visualisation's stability. Both modes also have filtering, row count limiting, and chart truncation options which reduce the amount of visualised data and keep the application stable. However, we will continue to investigate the limits of the application by experimenting on even larger databases with significant volumes of data.

Extensibility of the application

If VizER were to be extended, the architecture is separated enough such that further work could be easily integrated without breaking other components. For adding new visualisation types, for any charting library, only the following is required:

- Create a new component implementing the chart in the frontend.
- Add the option to the backend by adding it to its schema pattern's group, and add any extra conditions for recommendation (e.g. how many scalar attributes are needed, if any).

The amCharts library has a vast range of visualisations which VizER does not support. There is the opportunity to implement completely new visualisation types such as pie charts and donut charts, and use the characteristics of the chart to deduce what schema pattern they belong to. There is also the opportunity to implement variations of currently implemented visualisations. An example of this is choropleth maps. Currently, VizER can only visualise data on countries around the world. However, amCharts also has choropleth maps for specific countries like the USA - this could be used to visualise information about specific states. Therefore, there is a significant opportunity to widen the application's range of visualisation options, by including new chart types and new charting libraries.

5.1.5 User Experience

This section discusses the findings from user testing done on the application. There were two participant types for these user tests: expert users, who had experience with data analysis and working with Tableau, and regular, non-technical users. Each user was asked to perform the following tasks on Tableau first and then on VizER:

- Visualising the areas of continents.
- Visualising the inflation of countries against their unemployment rates.
- Visualising the elevation of airports on islands.
- Visualising the border lengths between countries where the length exceeds 2500km.

After those, users were asked to use the VIZ-FIRST mode to complete the following tasks:

- Create a scatter chart from the Economy table.
- Create a circle packing diagram from the Island table.

"VizER is easier to navigate than Tableau"

Overall, users felt that the UI for VizER was more intuitive than Tableau. Non-technical users felt overwhelmed by Tableau's complex interface due to the many components on the screen from the start. They believed Tableau would require time and research to become proficient and struggled with simple tasks like assigning columns to the right shelf. Users felt VizER gave a clearer path to generate visualisations. A pain point for regular users was adding filters in Tableau for the borders task. While the "Filters" box was on the screen, interacting with it gave no option to create a filter; rather, users had to search each top menu until finding the Filters option. Users preferred VizER's functionality of presenting the user with the option to apply a filter upon selecting the column.

However, some non-technical users still felt VizER was not intuitive enough. For example, they felt the GENERATE button rendering upon loading the page was misleading, and initially felt unclear about its functionality. Also, Tableau's drag-and-drop functionality was preferred to a checkbox interface for selecting columns. Some suggested improvements were:

- A step-by-step tutorial when first loading the application, which can be skipped by regular users of the application.
- Each step in the process should appear one after the other e.g. the GENERATE button should only appear after columns have been selected and filters have been applied.
- There should be more information about what each visualisation requires, similar to how Tableau's Show Me window has information about the type of columns needed to produce each chart and how many of each type.

"VizER provided better visualisations that made more sense"

Users, both expert and non-technical, felt recommendations made by VizER were better than Tableau. This was particularly prevalent with the borders task - users felt all visualisations offered by Tableau for this task were not meaningful and did not express the outcome of the task which was visualising the **length** of borders between countries. The chord diagram recommended by VizER was described as more suitable for the task and looked better than any of Tableau's choices. Users also enjoyed the range of visualisations offered by VizER, whereas they felt Tableau had a lot of similar-looking visualisation types. They felt the VIZ-FIRST mode was particularly useful for exploring all the chart types offered by the application and understanding what data was required to produce each one. All users appreciated the interactiveness of VizER's visualisations and preferred this to Tableau, which did not provide any options to zoom in or navigate charts. Specifically for the economy task, users disliked how clustered Tableau's scatter graph was with no option to spread the points out, whereas VizER gave the users zooming functionality with their mouse or the scrollbars on the chart.

"VizER still has room for improvement"

- While most users preferred VizER as a data visualiser over Tableau for the core services provided, they felt Tableau could offer more if they took time to master it. Some non-technical users struggled with understanding the exact step-by-step process of using the app to visualise their data. They felt VizER would benefit from labels to explain each step e.g. above "Tables" section could say "Step 1: Choose columns to visualise", then above the GENERATE section could say "Step 2: Apply filters then press GENERATE". While VizER was simpler to navigate than Tableau, users still felt like it could be even simpler for non-technical users with little to no experience of data analysis.

- Users enjoyed the VIZ-FIRST mode and felt using it first would have helped them understand how to use the DATA-FIRST mode better since it identified the suitable data for each visualisation type. However, if the purpose of VIZ-FIRST is an exploratory experience, there should be more guidance on what each visualisation type needs and why the tables shown meet those requirements.

- Non-technical users who were unfamiliar with ER models believed that more information could be given on the identified schema pattern and what relation it had with the recommended visualisations. Perhaps there could be an icon on the schema pattern component which when hovered over, would provide extensive information about why the chosen columns corresponded to that pattern, and what impact that had on which visualisations were recommended.

- Expert users felt that while VizER did well to provide complex visualisation types like chord and sankey diagrams which Tableau did not offer, but lacked the ability to perform complex queries on the data. A suggestion made for the DATA-FIRST mode was an SQL query input where instead of selecting columns, they could execute an SQL query directly and observe what pattern and visualisations were recommended as a result. Having this functionality would also mean technical users could apply more complex filters on the data. Similar to this, expert users felt that the application could benefit from features which gave the user more freedom such as choice of aggregation method (SUM, AVG, MAX etc.) or visualisation of data from multiple tables via table joins.

Ethical Considerations

The main ethical consideration for this project was the processing of user's data. The VizER application establishes its own connection to the user's database and gains access to all data within the database and the associated metadata. Therefore, we had to consider that the application could be granted access to data that is personal and/or sensitive. This is not necessarily a major issue as firstly, permission is granted by the user through entering their credentials, and secondly, none of the user's data from their database is processed anywhere outside the application itself.
A similar consideration we made for this project was the processing of the user's credentials. In order to create a connection the user's PostgreSQL database, the following information is required: username, password, host, port, and database name. The consideration we made was particularly to do with processing the user's password. We ensure that the user's password is censored on the screen (unless they wish to see it by clicking the eye icon) in the interest of user security. We also do not save the user's password anywhere.
The final ethical consideration made during this project was with the user testing. Testing was done one-on-one and in-person. Each participant was asked for consent to partake in user testing beforehand. Each user was briefed on what the testing would involve and what would be required from the user. No personal data of the user was recorded such as video or voice recordings.

Future Work

There are many potential extensions to the VizER application, whether the aim is to improve existent functionalities of the application, or introduce new features. In this section, we discuss what future work could be done with this project and the VizER application based on insights gained during the development and evaluation process, including the findings of our user testing.

Adding more diverse and detailed visualisation types

As discussed in Section 5.1.4, there is no limit to the range of visualisation types available across all charting libraries. Based on the limited time for this project, only the visualisations discussed in McBrien's paper [5] were implemented, with the addition of a force-directed network chart. Since our application is extensible enough to easily implement new chart types, we could explore amCharts for more unique and complex visualisation types, or research other charting libraries. An approach we can take is to navigate through different charting libraries for every visualisation type and choose the best for each chart type so that VizER truly offers the best visualisations with no dependence on a single library.
Currently, our recommendation process only uses the mandatory conditions for each visualisation type but McBrien also discusses optional conditions for each visualisation [5]. An example is the bubble chart - three scalar attributes are required as a mandatory condition, but a fourth attribute could also be visualised as the colour dimension for each bubble on the graph. For most visualisations, there is an optional attribute that can be selected to dictate the colour dimension for the graph. Integrating this functionality would allow VizER to produce more diverse and aesthetic visualisations that can visualise more data.

Visualising data from more than one table

At the moment, VizER only allows the visualisation of data from a single table in both modes of operation. We could extend the DATA-FIRST mode by allowing data to be visualised from multiple tables through the use of table joins. One potential approach to implement this in the backend would be to analyse each selected table and its foreign key(s) and determine a chain of foreign keys that will allow them all to be joined. Since our method for identifying the schema pattern is through analysis of the selected columns, joining tables should not affect that process. Having this functionality would allow the application to visualise a wider range of complex relationships between different tables.

Completeness - enforcing it instead of checking for it

At the moment, when a stacked bar chart or spider chart is considered, we check the completeness of the weak entity using the method described in Section 4.4.1. Recall that completeness is the property of a weak entity to have the same set of child key values for every parent key value. An alternative approach we could use for these chart types is enforcing completeness instead of checking for it. Instead of checking that our weak entity is complete, we could abstain from visualising records where the child key value was not common for all parent key values.

We would not recommend these chart types if all the parent key values did not have a single common child key value. This way, the user is still able to visualise their weak entity in a stacked bar or spider chart format but there is the chance that not **all** of their chosen data will be visualised (this situation should be communicated to the user).

Improvements to the UI

From the findings of our user testing, we received multiple suggestions on how to improve VizER's user interface or overall experience. For the DATA-FIRST mode, we concluded that further work could be done to experiment with different methods of selecting data to visualise and which methods were preferred by users. At the moment, a checkbox interface is used to manually select which columns the user would like to visualise. Implementing a drag-and-drop interface like Tableau could be more enjoyable and simple for users, and also allows users to set specific axes/fields (currently, the order of selected attributes dictates what each axis/field represents). Another interface to consider is an SQL query input box. This would allow users to implement their own queries and observe the corresponding schema pattern and recommended visualisation for their query's result. This would also enable technical users to create advanced visualisations that display complex relationships within the data. Having an SQL query as the data input as opposed to a selection of columns would not change our method for DATA-FIRST - we could perform the SQL query **first** to ensure the query is valid and produces results, then perform column analysis on the query result columns. Lastly, giving users more freedom over the characteristics of their visualisation such as choosing the aggregation method (SUM, AVG, MAX etc.) would introduce more variation in the possible visualisations they can create.

Widening the databases that can be connected to

Currently, the application is only able to connect to relational databases that use PostgreSQL DBMS. This was done as all databases used throughout development and testing were Postgres databases. Further work could be done to remove this restriction and allow the application to connect to relational databases using other DBMS such as MySQL, Oracle etc. This task would not be too difficult as it would just involve integrating the necessary JDBC drivers and adding an input to the "Connect to Database" form so users can select their database type. Once a database type is set and the request is sent, the backend would formulate the correct connection URL.

Conclusion

In this paper, we have explored widely used data visualisation tools and observed that none use the conceptual model of the data when making chart recommendations. We have proposed our own approach of identifying visualisation schema patterns and using pre-defined mappings between each schema pattern and visualisation groups to provide users with the best choice of recommended charts. By analysing the column selection to determine the corresponding ER representation, we can use the underlying conceptual model of the data to create both complex and meaningful visualisations.

We implemented an application VizER which uses the pre-defined mappings between schema patterns and their distinct groups of visualisations in two different ways:

- Firstly, the DATA-FIRST mode enables users to select the columns they wish to visualise, and recommends them specific chart types.
- Secondly, the VIZ-FIRST mode enables users to choose the chart type they want, and explores all possible visualisation options from a suitable table.

This way, we are able to handle both cases of data analysis: VIZ-FIRST for open-ended exploration, and DATA-FIRST for targeted analysis. Hence, we have created a well-rounded tool that makes accurate visualisation recommendations based on the user's data selection. We have also ensured that the application is simple and intuitive enough to be used by non-technical users. as well as expert users, who appreciated the lack of complexity required to create any visualisation compared to industry tools used today.

In terms of further work that can be done on the application, there are multiple pathways for future developers to improve the service. We use our findings from the user testing in order to determine what improvements can be made to the application - these include an even simpler UI with more help-based functionalities guiding the user towards creating the best visualisation. Further extension of VizER also includes the implementation of more visualisation types. Small changes can also be made to widen the user base such as enabling users with non-Postgres databases to connect by altering the JDBC connection string (e.g. MySQL databases).

In conclusion, the VizER application from this project did achieve the objectives set out in Section 1.2, providing a solid foundation of conceptual-model-based visualisation recommendation. This application was able to be used without a steep learning curve, and was enjoyed by users of all levels of technical knowledge. If we continue to work on this project, we can gain further insights and find applications for data visualisations based on conceptual modelling in the real world.

Gallery

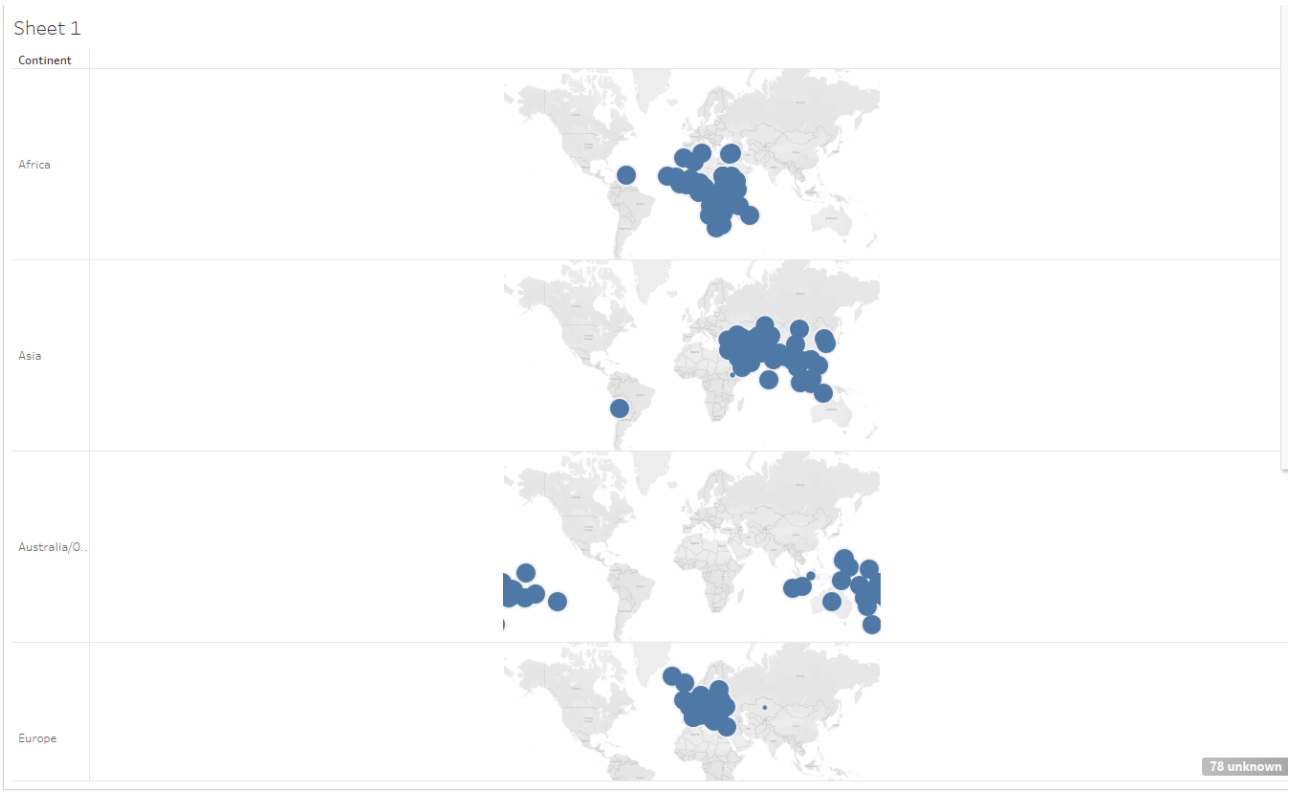


Figure A.1: Recommended visualisation from Tableau for the `encompasses` table

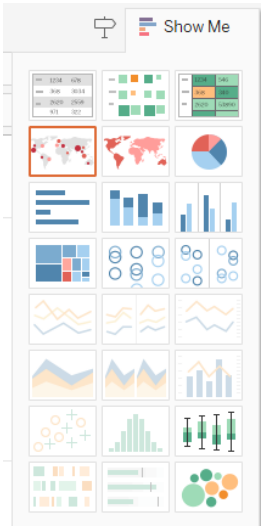


Figure A.2: Tableau's Show Me menu for the `encompasses` table

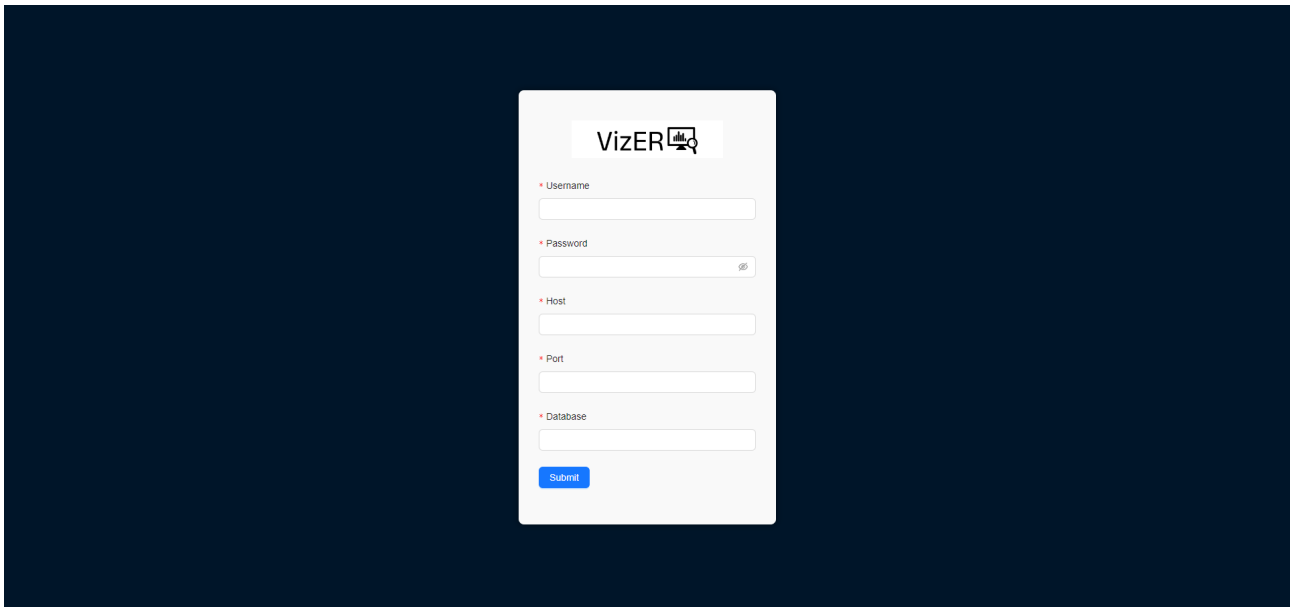


Figure A.3: "Connect to Database" page

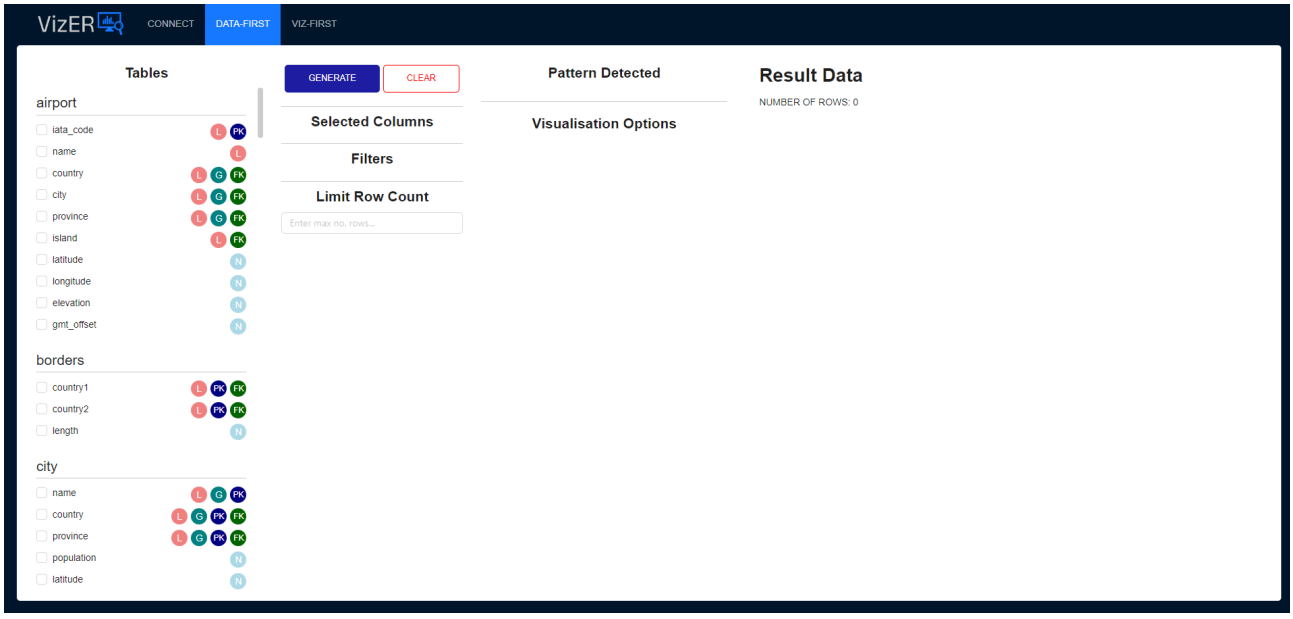


Figure A.4: DATA-FIRST page

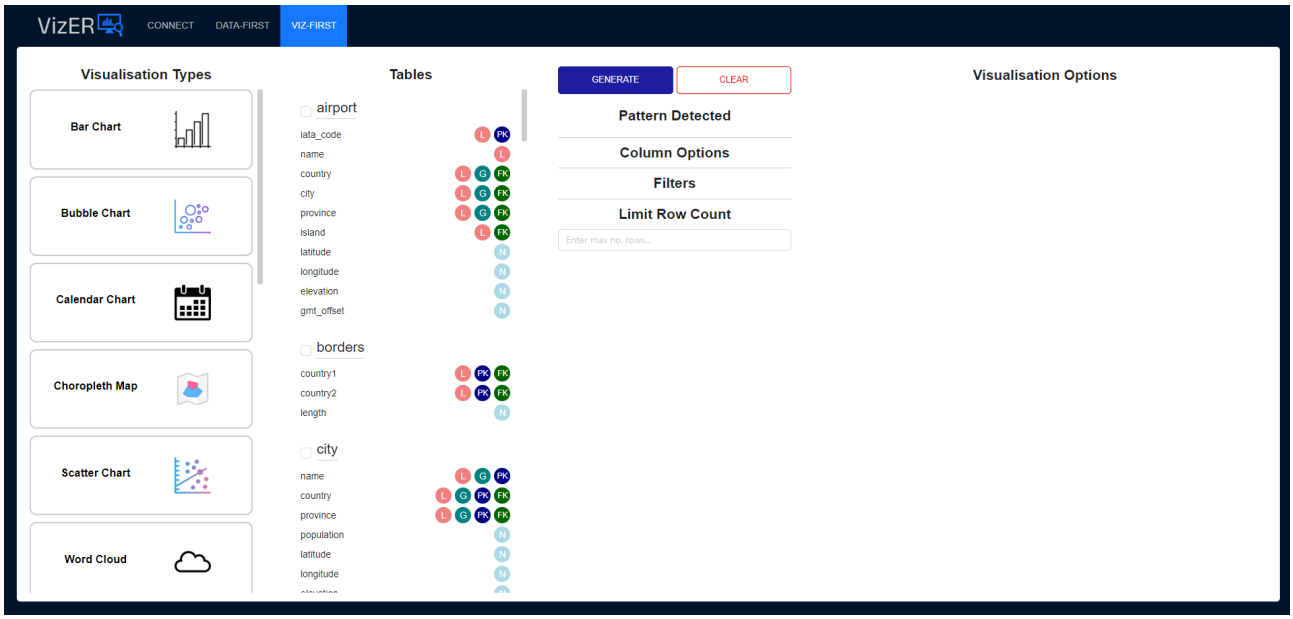


Figure A.5: VIZ-FIRST page

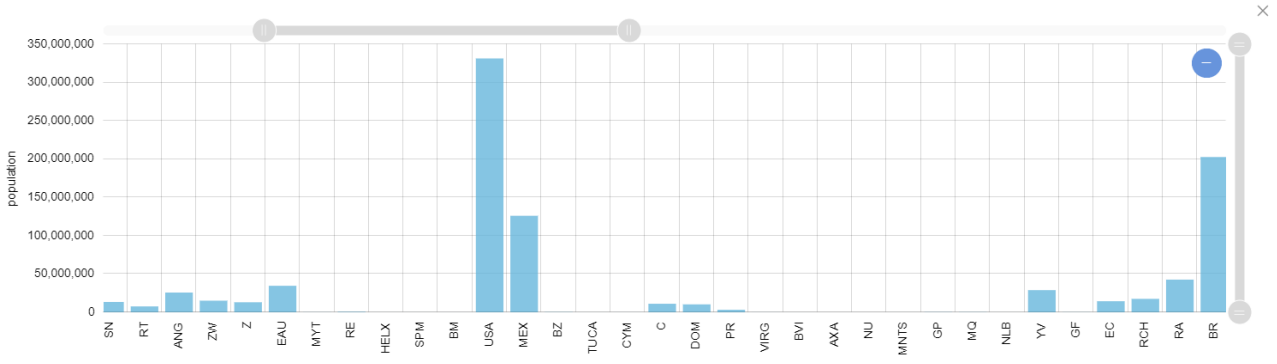


Figure A.6: Example bar chart for countries and their populations

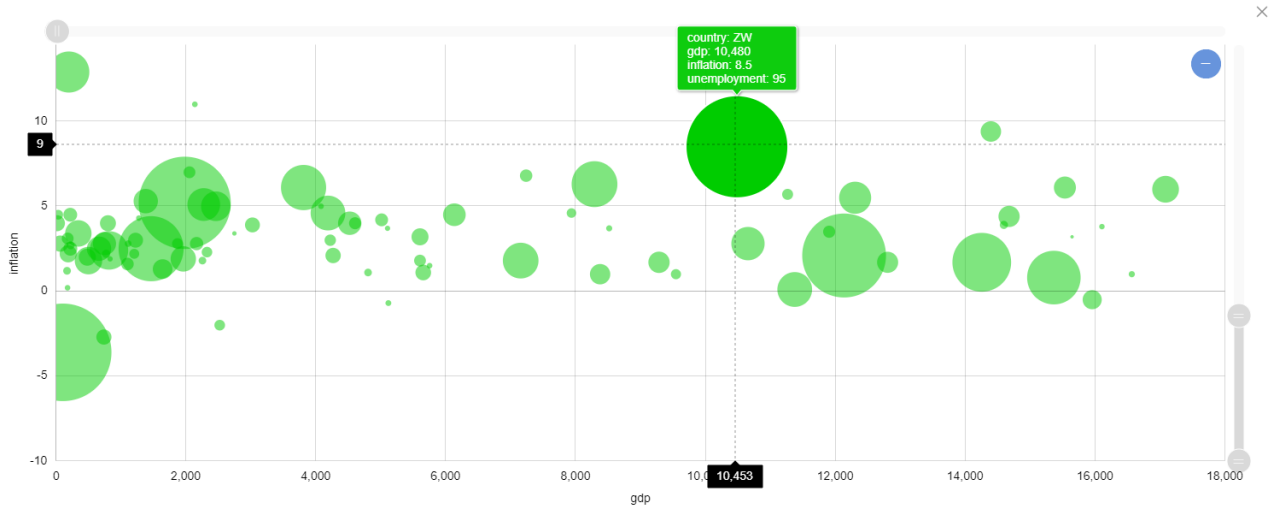


Figure A.7: Example bubble chart for gdp, inflation and unemployment figures of countries

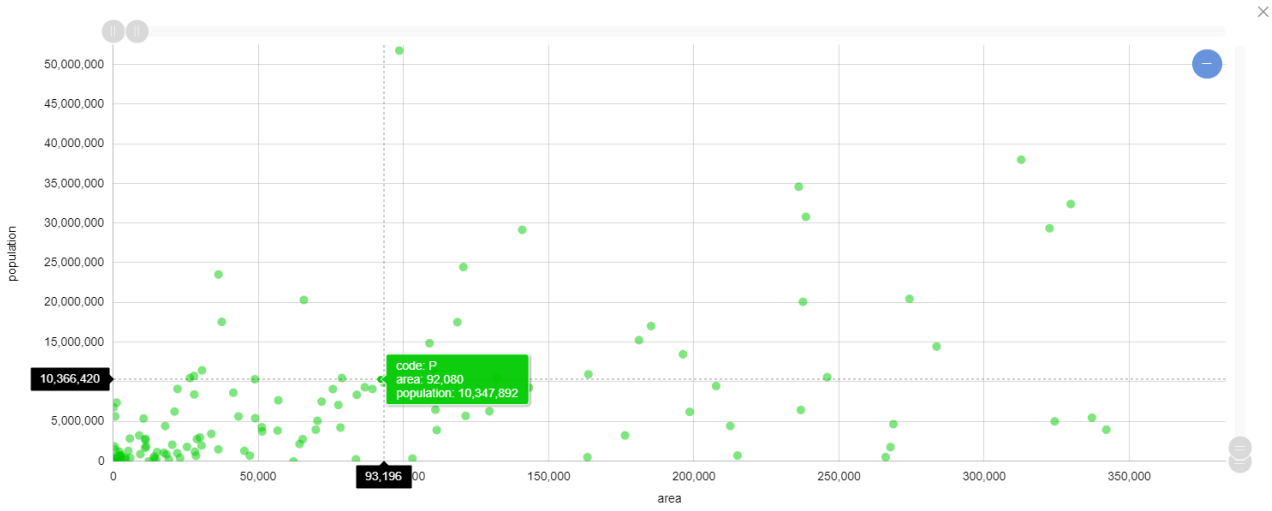


Figure A.8: Example scatter chart for countries' population vs. area

Figure A.9: Example word cloud for continents and their area

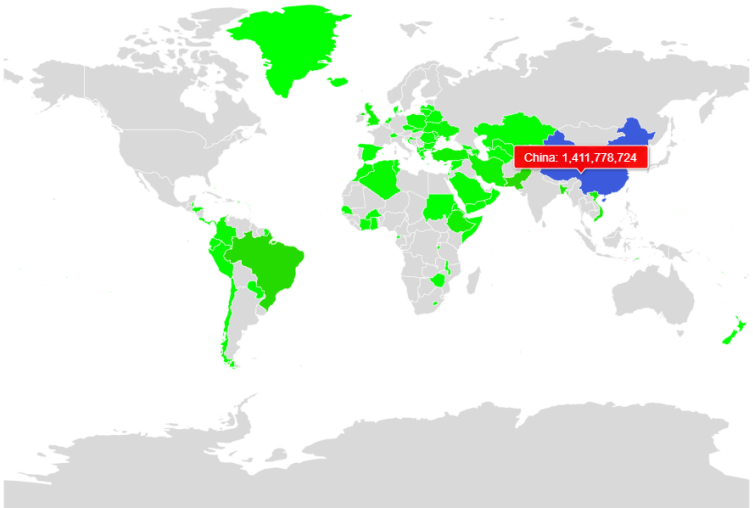


Figure A.10: Example choropleth map for countries and their populations

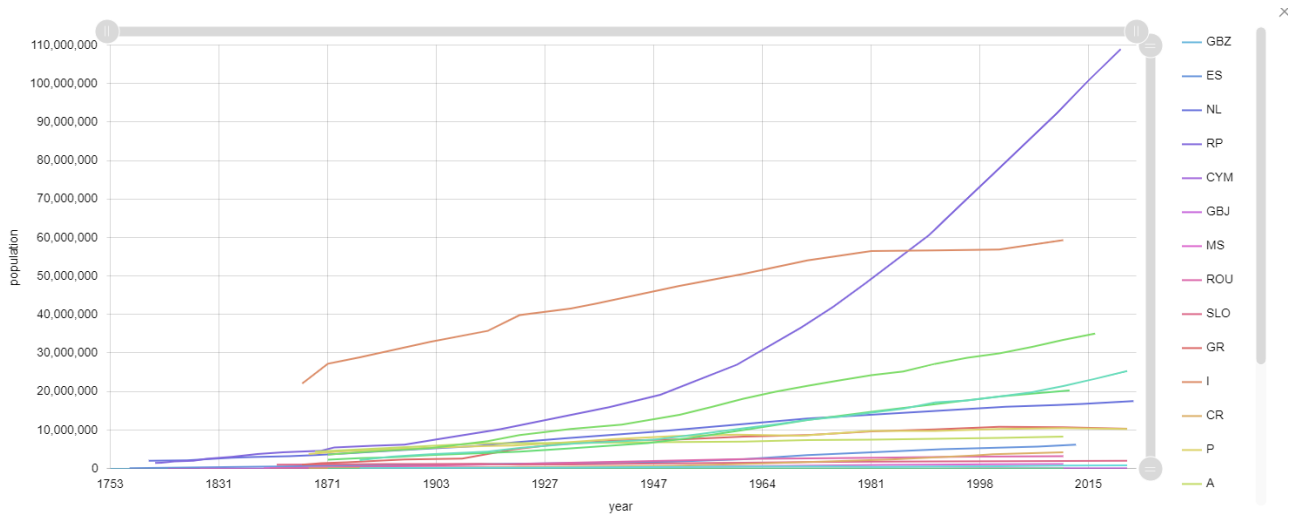


Figure A.11: Example line graph for countries' populations over time

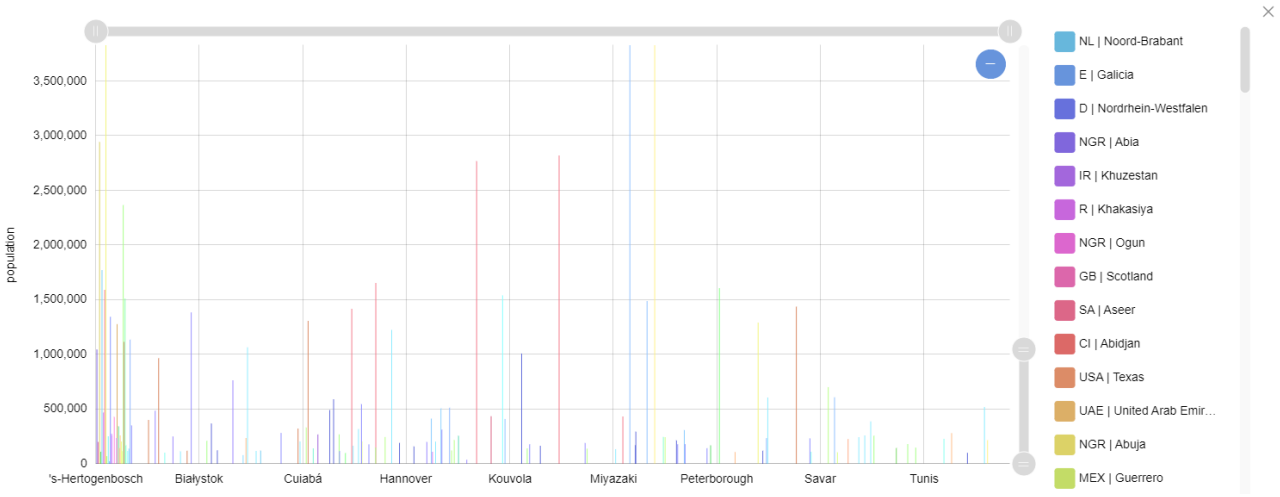


Figure A.12: Example grouped bar chart for countries' populatuions over time

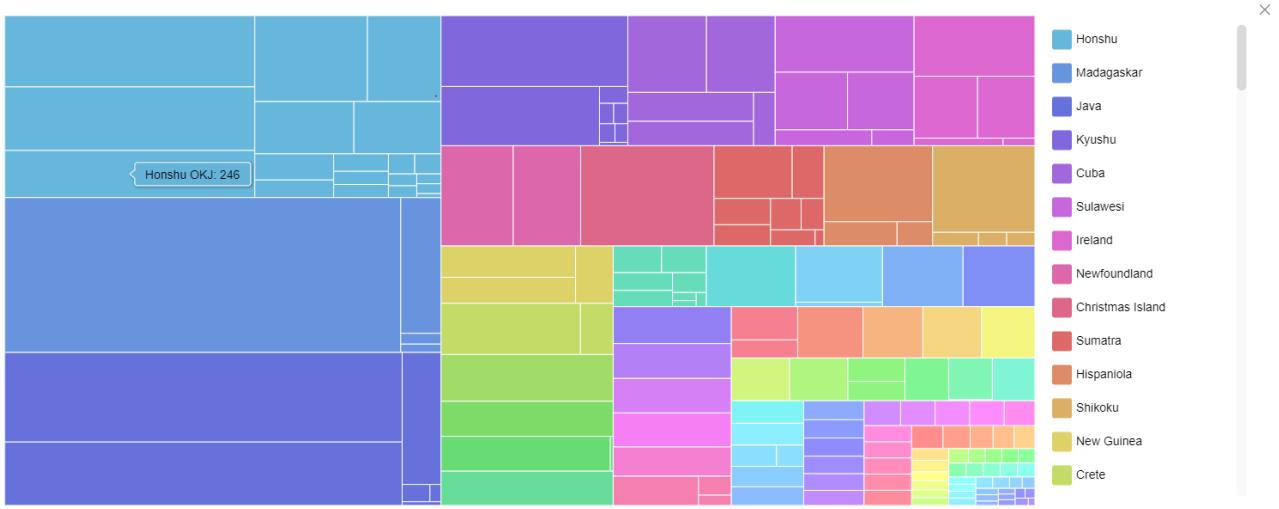


Figure A.13: Example treemap diagram for island airports and their elevation

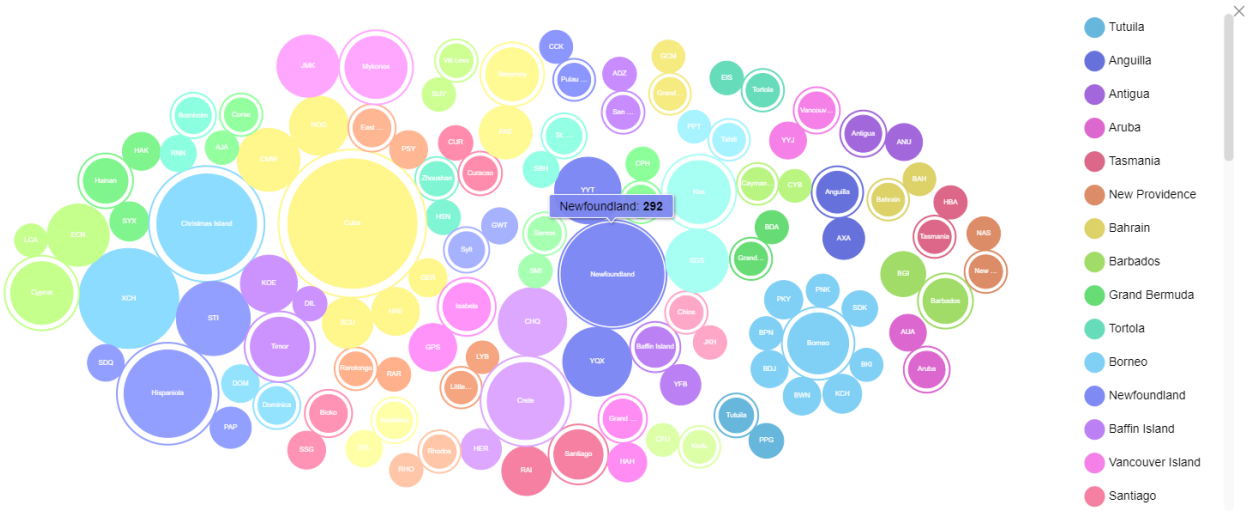


Figure A.14: Example circle packing diagram for island airports and their elevation

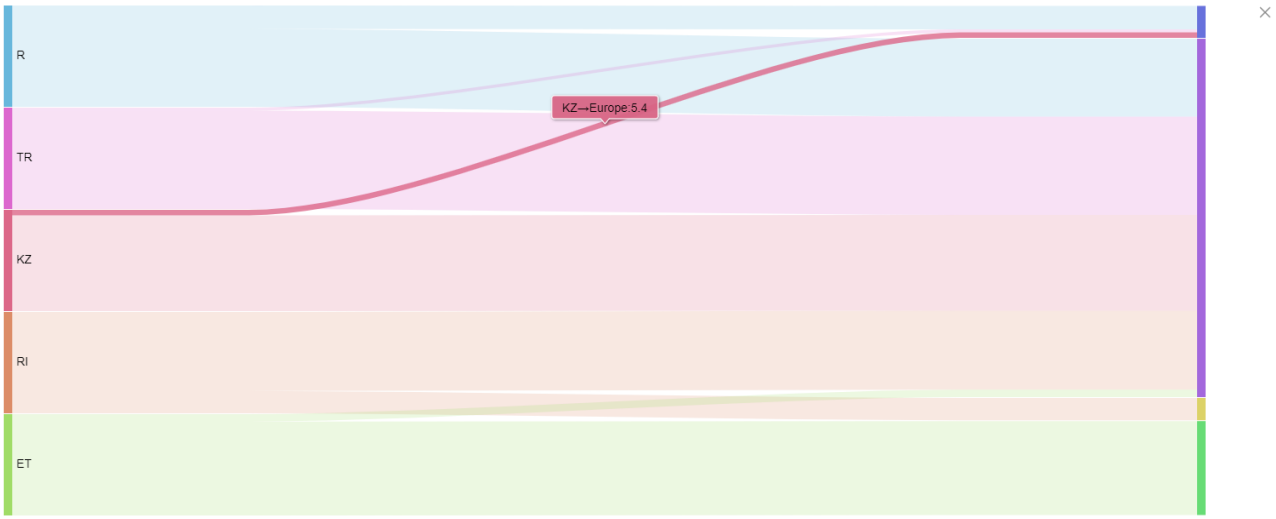


Figure A.15: Example sankey diagram for what percentage of a country is encompassed by its continent(s)

Bibliography

[1] Office of Research Integrity. Data Analysis. https://ori.hhs.gov/education/products/n_illinois_u/datamanagement/datopic.html. Accessed: June 17, 2024. pages 4

[2] Manasi Vartak, Sajjadur Rahman, Samuel Madden, Aditya Parameswaran, and Neoklis Polyzotis. Seedb: efficient data-driven visualization recommendations to support visual analytics. *Proc. VLDB Endow.*, 8(13):2182–2193, sep 2015. ISSN 2150-8097. doi: 10.14778/2831360.2831371. URL https://doi.org/10.14778/2831360.2831371. pages 4, 17

[3] Kanit Wongsuphasawat, Zening Qu, Dominik Moritz, Riley Chang, Felix Ouk, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Voyager 2: Augmenting visual analysis with partial view specifications. page 2648–2659, 2017. doi: 10.1145/3025453.3025768. URL https://doi.org/10.1145/3025453.3025768. pages 4, 18, 19

[4] Oskar Harmon Steven Batt, Tara Grealis and Paul Tomolonis. Learning tableau: A data visualization tool. *The Journal of Economic Education*, 51(3-4):317–328, 2020. doi: 10.1080/00220485.2020.1804503. URL https://doi.org/10.1080/00220485.2020.1804503. pages 4

[5] Peter Mc. Brien and Alexandra Poulovassilis. Towards data visualisation based on conceptual modelling and schema transformations automed technical report number 39. 2018. URL https://api.semanticscholar.org/CorpusID:198957663. pages 4, 5, 7, 11, 12, 14, 15, 19, 20, 21, 25, 26, 27, 28, 29, 32, 46, 58

[6] David Bull. Using database schemas and conceptual modelling to guide data visualisation, 2023. pages 5, 10, 50

[7] Colin Ware. *Information Visualization: Perception for Design: Second Edition*. 04 2004. pages 6, 7, 11

[8] T. Munzner. *Visualization Analysis and Design*. AK Peters Visualization Series. CRC Press, 2014. ISBN 9781498759717. URL https://books.google.de/books?id=NfkYCwAAQBAJ. pages 7

[9] L. Wilkinson, D. Wills, D. Rope, A. Norton, and R. Dubbs. *The Grammar of Graphics*. Statistics and Computing. Springer New York, 2005. ISBN 9780387245447. URL https://books.google.co.uk/books?id=_kRX4LoFfGQC. pages 7, 8

[10] Graham Wills and Leland Wilkinson. Autovis: Automatic visualization. *Information Visualization*, 9(1):47–69, 2010. doi: 10.1057/ivs.2008.27. URL https://doi.org/10.1057/ivs.2008.27. pages 7, 8, 16

[11] *GPL Reference Guide for IBM SPSS Statistics*. IBM, 2021. pages 8, 9

[12] H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Use R! Springer International Publishing, 2016. ISBN 9783319242750. URL https://books.google.co.uk/books?id=RTMFswEACAAJ. pages 9

[13] Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098. URL https://doi.org/10.1198/jcgs.2009.07098. pages 9

[14] Hadley Wickham. ggplot2. *WIREs Computational Statistics*, 3(2):180–185, 2011. doi: https://doi.org/10.1002/wics.147. URL https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/wics.147. pages 10

[15] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030. pages 10

[16] S. S. Stevens. On the theory of scales of measurement. *Science*, 103(2684):677–680, 1946. ISSN 00368075, 10959203. URL http://www.jstor.org/stable/1671815. pages 11, 12

[17] W.J. Premerlani and M.R. Blaha. An approach for reverse engineering of relational databases. pages 151–160, 1993. doi: 10.1109/WCRE.1993.287769. pages 12, 13

[18] Martin Andersson. Extracting an entity relationship schema from a relational database through reverse engineering. pages 403–419, 1994. pages 13

[19] J.-M. Petit, F. Toumani, J.-F. Boulicaut, and J. Kouloumdjian. Towards the reverse engineering of renormalized relational databases. pages 218–227, 1996. doi: 10.1109/ICDE.1996.492110. pages 14

[20] Erkin Salih. Data visualisation through conceptual model generation and analysis, 2019. pages 14

[21] Jock Mackinlay, Pat Hanrahan, and Chris Stolte. Show me: Automatic presentation for visual analysis. *IEEE transactions on visualization and computer graphics*, 13:1137–44, 11 2007. doi: 10.1109/TVCG.2007.70594. pages 14, 15

[22] URL https://www.tableau.com/en-gb/why-tableau/what-is-tableau. pages 14

[23] Kanit Wongsuphasawat, Dominik Moritz, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016. doi: 10.1109/TVCG.2015.2467191. pages 14, 18

[24] Leland Wilkinson, A. Anand, and Robert Grossman. Graph-theoretic scagnostics. *INFOVIS*, 5:157– 164, 11 2005. doi: 10.1109/INFVIS.2005.1532142. pages 16

[25] Manasi Vartak, Samuel Madden, Aditya Parameswaran, and Neoklis Polyzotis. Seedb: automatically generating query visualizations. *Proc. VLDB Endow.*, 7(13):1581–1584, aug 2014. ISSN 2150-8097. doi: 10.14778/2733004.2733035. URL https://doi.org/10.14778/2733004.2733035. pages 17

[26] Aditya Parameswaran, Neoklis Polyzotis, and Hector Garcia-Molina. Seedb: Visualizing database queries efficiently. *Proceedings of the VLDB Endowment*, 7:325–328, 12 2013. doi: 10.14778/2732240.2732250. pages 17